

Integrated autonomous controller

[Original code](#) / [PDF reference copy](#)

SOURCE INTEGRITY

Original filename: Pasted text(9).txt

Source lines: 2,995 | Source bytes: 116,365

SHA-256 (original bytes):

17946328c71838900b53a7bd0b00c42e108eed12c85cffc4577815e22ecd8b15

The following pages reproduce the supplied C++ source without changing its logic, comments, constants, or ordering. Long lines are wrapped for readability only.

The exact original text file is also embedded as a PDF attachment.

The listing is a source-code record, not a claim of physical hardware validation.

Integrated autonomous controller

ORIGINAL LINES 1-39

```
1  #include <Wire.h>
2  #include <math.h>
3  #include <HardwareSerial.h>
4  #include <stdio.h>
5  HardwareSerial DebugSerial(PA10, PA9);
6  // Raspberry Pi vision UART:
7  // RX=PB11, TX=PB10
8  // Physical wiring:
9  // Pi TX (GPIO14 / pin 8)  -> STM32 PB11 (RX3)
10 // Pi RX (GPIO15 / pin 10) -> STM32 PB10 (TX3)
11 // Pi GND                  -> STM32 GND
12 // Packet:
13 // V,<steerPct>,<speedPct>,<confidencePct>\n
14 // Example:
15 // V,35,80,92
16 // = look ahead RIGHT 35%, request 80% speed,
17 //   with 92% path confidence.
18 HardwareSerial PiSerial(PB11, PB10);
19 // =====
20 // ROBO RUMBLE - INTEGRATED AUTONOMOUS CONTROLLER
21 // FIXED I2C VERSION
22 // =====
23 // =====
24 // MOTOR DRIVER
25 // =====
26 #define PWMA  PA8
27 #define AIN1  PB12
28 #define AIN2  PB13
29 #define PWMB  PB0
30 #define BIN1  PB14
31 #define BIN2  PB15
32 #define STBY  PB1
33 // =====
34 // EMERGENCY STOP INPUT
35 // =====
36 // Wokwi simulation:
37 // PB5 uses INPUT_PULLUP.
38 // Button released = HIGH
39 // Button pressed  = LOW
```

Integrated autonomous controller

ORIGINAL LINES 40-78

```
40 // IMPORTANT FOR THE PHYSICAL ROBOT:
41 // The real mushroom E-Stop must ALSO interrupt motor
42 // power directly. The GPIO input is only for status/
43 // software awareness; it is not the primary safety cut.
44 // =====
45 #define ESTOP_PIN  PB5
46 // =====
47 // MAIN ON/OFF SWITCH INPUT
48 // =====
49 // Wokwi simulation:
50 // Use a LATCHING slide/rocker switch, not a momentary button.
51 // PA15 uses INPUT_PULLUP:
52 //   ON  = switch connects PA15 to GND -> LOW
53 //   OFF = switch open                -> HIGH
54 // A broken/open signal is therefore interpreted as OFF.
55 // IMPORTANT FOR THE PHYSICAL ROBOT:
56 // The real accessible main ON/OFF switch should interrupt
57 // the robot's main electrical supply. This GPIO is only a
58 // Wokwi simulation/status input. The E-Stop remains separate.
59 // =====
60 #define MAIN_SWITCH_PIN  PA15
61 // =====
62 // ENCODERS
63 // =====
64 #define LEFT_ENC_A  PB8
65 #define LEFT_ENC_B  PB9
66 // PB10/PB11 are reserved for Raspberry Pi UART.
67 // Right encoder uses PA11/PA12.
68 #define RIGHT_ENC_A  PA11
69 #define RIGHT_ENC_B  PA12
70 // =====
71 // VL53L1X FRONT COLLISION SENSORS
72 // =====
73 // The two ToF sensors are retained because other race cars
74 // and track walls can become collision hazards.
75 // Shared I2C:
76 //   SCL -> PB6
77 //   SDA -> PB7
78 // Individual XSHUT:
```

Integrated autonomous controller

ORIGINAL LINES 79-117

```
79 // Left -> PB3
80 // Right -> PB4
81 // Wokwi behavioural addresses:
82 // Left -> 0x30
83 // Right -> 0x31
84 // NOTE FOR PHYSICAL HARDWARE:
85 // Real VL53L1X modules normally boot at the same default
86 // address. The final physical firmware must bring them up
87 // one at a time with XSHUT and assign unique addresses.
88 // =====
89 #define LEFT_TOF_ADDR 0x30
90 #define RIGHT_TOF_ADDR 0x31
91 #define LEFT_XSHUT PB3
92 #define RIGHT_XSHUT PB4
93 #define SIM_RANGE_REGISTER 0x0096
94 // =====
95 // MPU6050
96 // =====
97 #define MPU_ADDR 0x68
98 #define MPU_PWR_MGMT_1 0x6B
99 #define MPU_GYRO_CONFIG 0x1B
100 #define MPU_ACCEL_XOUT_H 0x3B
101 #define MPU_GYRO_ZOUT_H 0x47
102 #define MPU_WHO_AM_I 0x75
103 const float GYRO_SENSITIVITY = 131.0f;
104 // =====
105 // IR ARRAY
106 // =====
107 const uint8_t IR_PINS[8] = {
108     PA0, PA1, PA2, PA3, PA4, PA5, PA6, PA7
109 };
110 // =====
111 // ROAD-BOUNDARY INTERPRETATION
112 // =====
113 // IMPORTANT:
114 // HIGH now means an IR sensor is seeing a ROAD BOUNDARY,
115 // not a centre line.
116 // Sensor layout:
117 // IR1 IR2 IR3 IR4 | IR5 IR6 IR7 IR8
```

Integrated autonomous controller

ORIGINAL LINES 118-156

```
118 //    LEFT SIDE      |      RIGHT SIDE
119 // The weights are deliberately strongest near the
120 // centre of the robot. If a boundary reaches IR4 or IR5,
121 // the robot is close to crossing it and must turn away
122 // aggressively.
123 // Positive value  -> steer RIGHT (boundary on the left)
124 // Negative value  -> steer LEFT  (boundary on the right)
125 // Example:
126 // 00000000 -> road clear, drive straight
127 // 10000000 -> left outer boundary, gentle right
128 // 00100000 -> left inner boundary, stronger right
129 // 00000100 -> right inner boundary, stronger left
130 // 00000001 -> right outer boundary, gentle left
131 // =====
132 const float BOUNDARY_STEER_WEIGHTS[8] = {
133     1.0f, 2.0f, 3.0f, 4.0f, -4.0f, -3.0f, -2.0f, -1.0f
134 };
135 uint8_t sensor[8];
136 // =====
137 // ENVIRONMENT + SPEED PROFILES
138 // =====
139 // WOKWI:
140 //   RUNNING_IN_WOKWI = true
141 //   Keeps the already-tested ~38 RPM scaled motor model.
142 // PHYSICAL ROBOT:
143 //   RUNNING_IN_WOKWI = false
144 //   Start with:
145 //     PHYSICAL_INITIAL_TEST_PROFILE = true
146 //     Normal target = 600 RPM ~= 1.35 m/s with 43 mm wheels.
147 //   After real traction, braking, current and PID testing:
148 //     PHYSICAL_INITIAL_TEST_PROFILE = false
149 //     Normal straight target = 1050 RPM ~= 2.36 m/s.
150 // The faster profile is NOT permission to drive every corner
151 // at 2.36 m/s. Camera speed recommendation still reduces the
152 // target before curves, while IR/ToF/IMU safety can override it.
153 // =====
154 const bool RUNNING_IN_WOKWI = true;
155 // Used only when RUNNING_IN_WOKWI = false.
156 const bool PHYSICAL_INITIAL_TEST_PROFILE = true;
```

Integrated autonomous controller

ORIGINAL LINES 157-195

```
157 // -----
158 // WOKWI PROFILE
159 // -----
160 const float WOKWI_NORMAL_BASE_RPM = 38.0f;
161 const float WOKWI_CAUTION_BASE_RPM = 28.0f;
162 const float WOKWI_SENSOR_FAULT_RPM = 20.0f;
163 const float WOKWI_BOUNDARY_CAUTION_RPM = 32.0f;
164 const float WOKWI_BOUNDARY_INNER_RPM = 28.0f;
165 const float WOKWI_BOUNDARY_CENTRE_RPM = 20.0f;
166 const float WOKWI_SKID_BASE_RPM = 22.0f;
167 const float WOKWI_MIN_TARGET_RPM = 8.0f;
168 const float WOKWI_MAX_TARGET_RPM = 40.0f;
169 // -----
170 // PHYSICAL INITIAL TEST PROFILE
171 // -----
172 // 43 mm wheels:
173 //   450 RPM ~= 1.01 m/s
174 //   600 RPM ~= 1.35 m/s
175 // -----
176 const float PHYS_TEST_NORMAL_BASE_RPM = 600.0f;
177 const float PHYS_TEST_CAUTION_BASE_RPM = 450.0f;
178 const float PHYS_TEST_SENSOR_FAULT_RPM = 350.0f;
179 const float PHYS_TEST_BOUNDARY_CAUTION_RPM = 500.0f;
180 const float PHYS_TEST_BOUNDARY_INNER_RPM = 420.0f;
181 const float PHYS_TEST_BOUNDARY_CENTRE_RPM = 320.0f;
182 const float PHYS_TEST_SKID_BASE_RPM = 350.0f;
183 const float PHYS_TEST_MIN_TARGET_RPM = 120.0f;
184 const float PHYS_TEST_MAX_TARGET_RPM = 650.0f;
185 // -----
186 // PHYSICAL RACE PROFILE
187 // -----
188 // 43 mm wheels:
189 //   450 RPM ~= 1.01 m/s
190 //   525 RPM ~= 1.18 m/s
191 //   735 RPM ~= 1.66 m/s
192 //   1050 RPM ~= 2.36 m/s
193 // 1050 RPM is the first high-performance straight target.
194 // Speeds toward 3 m/s remain a later physical-test goal,
195 // not a default continuous race speed.
```

Integrated autonomous controller

ORIGINAL LINES 196-230

```
196 // -----
197 const float PHYS_RACE_NORMAL_BASE_RPM = 1050.0f;
198 const float PHYS_RACE_CAUTION_BASE_RPM = 700.0f;
199 const float PHYS_RACE_SENSOR_FAULT_RPM = 500.0f;
200 const float PHYS_RACE_BOUNDARY_CAUTION_RPM = 800.0f;
201 const float PHYS_RACE_BOUNDARY_INNER_RPM = 650.0f;
202 const float PHYS_RACE_BOUNDARY_CENTRE_RPM = 500.0f;
203 const float PHYS_RACE_SKID_BASE_RPM = 500.0f;
204 const float PHYS_RACE_MIN_TARGET_RPM = 180.0f;
205 // Allows differential steering margin above the 1050 RPM base.
206 const float PHYS_RACE_MAX_TARGET_RPM = 1150.0f;
207 // -----
208 // ACTIVE PROFILE
209 // -----
210 const float NORMAL_BASE_RPM = RUNNING_IN_WOKWI ? WOKWI_NORMAL_BASE_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_NORMAL_BASE_RPM
211 : PHYS_RACE_NORMAL_BASE_RPM );
212 const float CAUTION_BASE_RPM = RUNNING_IN_WOKWI ? WOKWI_CAUTION_BASE_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_CAUTION_BASE_RPM
213 : PHYS_RACE_CAUTION_BASE_RPM );
214 const float SENSOR_FAULT_RPM = RUNNING_IN_WOKWI ? WOKWI_SENSOR_FAULT_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_SENSOR_FAULT_RPM
215 : PHYS_RACE_SENSOR_FAULT_RPM );
216 const float BOUNDARY_CAUTION_RPM = RUNNING_IN_WOKWI ? WOKWI_BOUNDARY_CAUTION_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE
217 ? PHYS_TEST_BOUNDARY_CAUTION_RPM : PHYS_RACE_BOUNDARY_CAUTION_RPM );
218 const float BOUNDARY_INNER_RPM = RUNNING_IN_WOKWI ? WOKWI_BOUNDARY_INNER_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_BOUNDARY_IN
219 : PHYS_RACE_BOUNDARY_INNER_RPM );
220 const float BOUNDARY_CENTRE_RPM = RUNNING_IN_WOKWI ? WOKWI_BOUNDARY_CENTRE_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_BOUNDARY_
221 : PHYS_RACE_BOUNDARY_CENTRE_RPM );
222 const float SKID_BASE_RPM = RUNNING_IN_WOKWI ? WOKWI_SKID_BASE_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_SKID_BASE_RPM
223 : PHYS_RACE_SKID_BASE_RPM );
224 const float MIN_TARGET_RPM = RUNNING_IN_WOKWI ? WOKWI_MIN_TARGET_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_MIN_TARGET_RPM
225 : PHYS_RACE_MIN_TARGET_RPM );
226 const float MAX_TARGET_RPM = RUNNING_IN_WOKWI ? WOKWI_MAX_TARGET_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_MAX_TARGET_RPM
227 : PHYS_RACE_MAX_TARGET_RPM );
228 // Steering gain is scaled to the active RPM range.
229 const float K_BOUNDARY_STEER = RUNNING_IN_WOKWI ? 3.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 45.0f : 75.0f );
230 // =====
```

Integrated autonomous controller

ORIGINAL LINES 231-269

```
231 // RACE CONTROL TIMING
232 // =====
233 // Boundary control is deliberately much faster than the
234 // motor-speed PID. This lets the robot react to the road
235 // edge quickly while the PID handles wheel-speed control.
236 // Wokwi:
237 //   Road/boundary loop = 10 ms (100 Hz)
238 //   Motor PID loop     = 100 ms (10 Hz)
239 // On the physical STM32, the IR/boundary loop can later
240 // be pushed much faster after testing.
241 // =====
242 const unsigned long ROAD_CONTROL_INTERVAL_MS = 10;
243 unsigned long previousRoadControlTime = 0;
244 // Other race cars and track walls are treated as
245 // collision hazards, so the ToF safety layer stays active.
246 const bool OBSTACLE_AVOIDANCE_ENABLED = true;
247 // =====
248 // WOKWI STABLE INTEGRATED SENSOR MODE
249 // =====
250 // Earlier subsystem tests already proved the MPU6050,
251 // VL53L1X and INA219 logic individually. In the large
252 // integrated Wokwi build, several I2C devices/custom chips
253 // on one simulated bus can occasionally lock the simulator.
254 // This switch keeps the REAL I2C code in this same file,
255 // but uses deterministic internal sensor values while
256 // finishing the integrated racing/state-machine tests.
257 // WOKWI:
258 //   true  = use internal MPU/ToF/INA values
259 // PHYSICAL ROBOT:
260 //   false = use the real I2C sensors on PB6/PB7
261 // This changes NO physical wiring and does NOT remove the
262 // sensors from the final design/documentation.
263 // =====
264 // =====
265 // WOKWI SENSOR-SOURCE SELECTION
266 // =====
267 // We want the VL53L1X DISTANCE SLIDERS to remain interactive,
268 // so ToF uses the real custom Wokwi I2C chips.
269 // The MPU6050 stays internally simulated for Wokwi stability.
```


Integrated autonomous controller

ORIGINAL LINES 270-308

```
270 // The INA219 now uses the interactive Wokwi slider/custom I2C
271 // chip so current and voltage can be changed live during tests.
272 // PHYSICAL ROBOT:
273 // Set all three internal-simulation flags to false.
274 // =====
275 // FALSE = read the two custom VL53L1X chips and their sliders.
276 const bool USE_WOKWI_INTERNAL_TOF_SIM = false;
277 const bool USE_WOKWI_TOF_SLIDER_CHIPS = RUNNING_IN_WOKWI;
278 // TRUE = use a deterministic gyro value in Wokwi.
279 const bool USE_WOKWI_INTERNAL_IMU_SIM = RUNNING_IN_WOKWI;
280 // FALSE = read the interactive INA219 Wokwi slider/custom I2C chip.
281 // On the physical robot this is also false, so the real INA219 is
282 // read over I2C at address 0x40.
283 const bool USE_WOKWI_INTERNAL_INA_SIM = false;
284 // Diagnostic/documentation flag for the current Wokwi build.
285 const bool USE_WOKWI_INA219_SLIDER_CHIP = RUNNING_IN_WOKWI;
286 // -----
287 // INTERNAL IMU TEST VALUES
288 // -----
289 const bool WOKWI_IMU_ONLINE = true;
290 const float WOKWI_GYRO_Z_DPS = 0.0f;
291 // =====
292 // AUTONOMOUS BRIDGE HANDLING
293 // =====
294 // Physical robot:
295 //   Pi Camera predicts the bridge/ramp ahead.
296 //   MPU6050 confirms chassis pitch.
297 //   STM32 selects bridge state and safe speed automatically.
298 // No human chooses a bridge state during the race.
299 // Wokwi:
300 // This dedicated test automatically creates one complete
301 // bridge profile after MAIN SWITCH is turned ON.
302 // =====
303 // Bridge control logic remains fully enabled.
304 // Only the automatic Wokwi bridge DEMO is disabled in this
305 // dedicated corner-speed test build because the bridge sequence
306 // has already been validated separately.
307 // FINAL INTEGRATED BUILD:
308 // Bridge logic remains active, but no scripted Wokwi bridge test runs.
```

Integrated autonomous controller

ORIGINAL LINES 309-347

```
309 const bool WOKWI_AUTO_BRIDGE_TEST = false;
310 const float WOKWI_MANUAL_PITCH_DEG = 0.0f;
311 const int WOKWI_MANUAL_BRIDGE_CONFIDENCE_PCT = 0;
312 // Automatic Wokwi timeline, measured from first ON state.
313 const unsigned long BRIDGE_TEST_FLAT_MS      = 3000;
314 const unsigned long BRIDGE_TEST_APPROACH_MS  = 5000;
315 const unsigned long BRIDGE_TEST_CLIMB_MS     = 8000;
316 const unsigned long BRIDGE_TEST_CRESCENT_MS  = 9500;
317 const unsigned long BRIDGE_TEST_DESCENT_MS   = 12500;
318 const unsigned long BRIDGE_TEST_EXIT_MS      = 14500;
319 // First-pass thresholds for a mild bridge slope.
320 // Retune these on the physical bridge after measuring
321 // real pitch noise and the actual bridge angle.
322 const int BRIDGE_CAMERA_ENTER_CONFIDENCE_PCT = 70;
323 const int BRIDGE_CAMERA_CLEAR_CONFIDENCE_PCT = 40;
324 const float BRIDGE_CLIMB_ENTER_PITCH_DEG = 2.0f;
325 const float BRIDGE_CRESCENT_PITCH_DEG = 1.2f;
326 const float BRIDGE_DESCENT_ENTER_PITCH_DEG = -2.0f;
327 const float BRIDGE_FLAT_PITCH_DEG = 1.0f;
328 const unsigned long BRIDGE_EXIT_CONFIRM_MS = 800;
329 // =====
330 // BRIDGE SPEED PROFILE
331 // =====
332 // Wokwi keeps the mild-slope values already tested.
333 // Physical initial:
334 //   normal 600
335 //   approach 540
336 //   climb 510
337 //   crest 480
338 //   descent 480
339 //   exit 540
340 // Physical race:
341 //   normal 1050
342 //   approach 900
343 //   climb 850
344 //   crest 800
345 //   descent 800
346 //   exit 900
347 // Final physical values must be tuned using the real bridge.
```

Integrated autonomous controller

ORIGINAL LINES 348-382

```
348 // =====
349 const float WOKWI_BRIDGE_APPROACH_RPM = 34.0f;
350 const float WOKWI_BRIDGE_CLIMB_RPM = 32.0f;
351 const float WOKWI_BRIDGE_CREST_RPM = 30.0f;
352 const float WOKWI_BRIDGE_DESCENT_RPM = 30.0f;
353 const float WOKWI_BRIDGE_EXIT_RPM = 34.0f;
354 const float PHYS_TEST_BRIDGE_APPROACH_RPM = 540.0f;
355 const float PHYS_TEST_BRIDGE_CLIMB_RPM = 510.0f;
356 const float PHYS_TEST_BRIDGE_CREST_RPM = 480.0f;
357 const float PHYS_TEST_BRIDGE_DESCENT_RPM = 480.0f;
358 const float PHYS_TEST_BRIDGE_EXIT_RPM = 540.0f;
359 const float PHYS_RACE_BRIDGE_APPROACH_RPM = 900.0f;
360 const float PHYS_RACE_BRIDGE_CLIMB_RPM = 850.0f;
361 const float PHYS_RACE_BRIDGE_CREST_RPM = 800.0f;
362 const float PHYS_RACE_BRIDGE_DESCENT_RPM = 800.0f;
363 const float PHYS_RACE_BRIDGE_EXIT_RPM = 900.0f;
364 const float BRIDGE_APPROACH_RPM = RUNNING_IN_WOKWI ? WOKWI_BRIDGE_APPROACH_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_BRIDGE_AP
PROACH_RPM
365 : PHYS_RACE_BRIDGE_APPROACH_RPM );
366 const float BRIDGE_CLIMB_RPM = RUNNING_IN_WOKWI ? WOKWI_BRIDGE_CLIMB_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_BRIDGE_CLIMB_RP
M
367 : PHYS_RACE_BRIDGE_CLIMB_RPM );
368 const float BRIDGE_CREST_RPM = RUNNING_IN_WOKWI ? WOKWI_BRIDGE_CREST_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_BRIDGE_CREST_RP
M
369 : PHYS_RACE_BRIDGE_CREST_RPM );
370 const float BRIDGE_DESCENT_RPM = RUNNING_IN_WOKWI ? WOKWI_BRIDGE_DESCENT_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_BRIDGE_DESC
ENT_RPM
371 : PHYS_RACE_BRIDGE_DESCENT_RPM );
372 const float BRIDGE_EXIT_RPM = RUNNING_IN_WOKWI ? WOKWI_BRIDGE_EXIT_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_BRIDGE_EXIT_RPM
373 : PHYS_RACE_BRIDGE_EXIT_RPM );
374 const float BRIDGE_APPROACH_STEER_RPM = RUNNING_IN_WOKWI ? 8.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 120.0f : 200.0f );
375 const float BRIDGE_CLIMB_STEER_RPM = RUNNING_IN_WOKWI ? 7.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 100.0f : 170.0f );
376 const float BRIDGE_CREST_STEER_RPM = RUNNING_IN_WOKWI ? 6.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 90.0f : 150.0f );
377 const float BRIDGE_DESCENT_STEER_RPM = RUNNING_IN_WOKWI ? 6.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 90.0f : 150.0f );
378 const float BRIDGE_EXIT_STEER_RPM = RUNNING_IN_WOKWI ? 8.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 120.0f : 200.0f );
379 float wokwiSimulatedPitchDeg = 0.0f;
380 int wokwiSimulatedBridgeConfidencePct = 0;
381 bool wokwiBridgeTestStarted = false;
382 unsigned long wokwiBridgeTestStartTime = 0;
```

Integrated autonomous controller

ORIGINAL LINES 383-421

```
383 // -----
384 // INTERNAL INA219 FALLBACK VALUES
385 // -----
386 // These values are used only if USE_WOKWI_INTERNAL_INA_SIM
387 // is changed back to true. In the current Wokwi build, the
388 // sliders control voltage and current live over I2C.
389 const bool WOKWI_INA_ONLINE = true;
390 const float WOKWI_BUS_VOLTAGE_V = 5.0f;
391 const float WOKWI_MOTOR_CURRENT_MA = 500.0f;
392 // =====
393 // RASPBERRY PI CAMERA / PREDICTIVE STEERING
394 // =====
395 // Camera/Pi:
396 //   sees the road ahead, predicts the next bend,
397 //   and recommends steering + approach speed.
398 // IR array:
399 //   reacts to the boundary directly under the car.
400 // STM32:
401 //   remains the final authority for motors, PID,
402 //   skid protection, stall protection and E-Stop.
403 // Packet from Pi:
404 //   V,<steerPct>,<speedPct>,<confidencePct>,<bridgeConfidencePct>
405 // steerPct:
406 //   -100 = hard LEFT
407 //    0 = straight
408 //   +100 = hard RIGHT
409 // speedPct:
410 //   50..100 recommended race speed
411 // confidencePct:
412 //   0..100 path-detection confidence
413 // =====
414 const unsigned long VISION_TIMEOUT_MS = 250;
415 // =====
416 // WOKWI INTERNAL CAMERA / PI SIMULATOR
417 // =====
418 // Wokwi cannot reproduce the real Raspberry Pi Camera
419 // processing pipeline. To keep testing reliable, this mode
420 // injects the SAME steering/speed/confidence command that the
421 // physical Raspberry Pi will later send over UART.
```

Integrated autonomous controller

ORIGINAL LINES 422-460

```
422 // Leave this TRUE while finishing the Wokwi controller.
423 // Set it FALSE on the physical robot so PB11 receives the
424 // real Raspberry Pi UART packets.
425 // FINAL INTEGRATED WOKWI VISION PLACEHOLDER:
426 // No scripted test is run automatically.
427 // The internal vision source starts neutral:
428 //   steering      = 0%
429 //   speed          = 100%
430 //   confidence     = 95%
431 //   bridge confidence = 0%
432 //   corner severity = 0%
433 // Therefore, after the operator deliberately starts the race,
434 // the default simulated road is simply a clear STRAIGHT.
435 // The normal IR, ToF, IMU, INA219, encoder, PID, bridge,
436 // corner-speed, stall, skid, E-Stop and fail-safe logic remains
437 // fully integrated and will react only when its inputs require it.
438 // =====
439 const bool USE_WOKWI_INTERNAL_VISION_SIM = RUNNING_IN_WOKWI;
440 // FINAL INTEGRATED BUILD:
441 // No scripted/automatic corner test is allowed to run.
442 const bool WOKWI_AUTO_CORNER_SPEED_TEST = false;
443 const int WOKWI_VISION_STEER_PCT = 0;
444 const int WOKWI_VISION_SPEED_PCT = 100;
445 const int WOKWI_VISION_CONFIDENCE_PCT = 95;
446 const int WOKWI_VISION_CORNER_SEVERITY_PCT = 0;
447 const bool WOKWI_VISION_ONLINE = true;
448 // Automatic corner-test timing.
449 // This test now has its OWN timer and does not depend on the
450 // bridge test at all.
451 //   0-3 s   : STRAIGHT
452 //   3-6 s   : GENTLE RIGHT
453 //   6-9 s   : MEDIUM LEFT
454 //   9-12 s  : SHARP RIGHT
455 //   12-15 s : VERY SHARP LEFT
456 //   >15 s   : STRAIGHT AGAIN
457 const unsigned long CORNER_TEST_GENTLE_MS = 3000;
458 const unsigned long CORNER_TEST_MEDIUM_MS = 6000;
459 const unsigned long CORNER_TEST_SHARP_MS = 9000;
460 const unsigned long CORNER_TEST_VERY_SHARP_MS = 12000;
```

Integrated autonomous controller

ORIGINAL LINES 461-499

```
461 const unsigned long CORNER_TEST_END_MS = 15000;
462 bool wokwiCornerTestStarted = false;
463 unsigned long wokwiCornerTestStartTime = 0;
464 const unsigned long WOKWI_VISION_INTERVAL_MS = 40;
465 unsigned long previousWokwiVisionTime = 0;
466 const int VISION_MIN_CONFIDENCE_PCT = 45;
467 const int VISION_MIN_SPEED_PCT = 50;
468 const int VISION_MAX_SPEED_PCT = 100;
469 const float VISION_MAX_STEER_RPM = RUNNING_IN_WOKWI ? 10.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 150.0f : 250.0f );
470 // Minimum speed the camera is allowed to request for a
471 // predicted sharp corner.
472 // Physical lower limit ~450 RPM ~= 1.0 m/s with 43 mm wheels.
473 const float VISION_MIN_BASE_RPM = RUNNING_IN_WOKWI ? 20.0f : 450.0f;
474 // Speed used if the camera is offline/stale/low-confidence.
475 const float VISION_FALLBACK_RPM = RUNNING_IN_WOKWI ? 28.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 450.0f : 550.0f );
476 // =====
477 // DYNAMIC CORNER-SPEED CONTROL
478 // =====
479 // Cornering force:
480 //      F = m * v^2 / R
481 // Because speed is squared, the car must slow down
482 // non-linearly as the upcoming road curvature increases.
483 // The Pi will estimate CORNER SEVERITY from upcoming road
484 // curvature. STM32 then applies an independent speed cap:
485 // speedFactor = 1 / sqrt(1 + K * severity)
486 // where severity is 0.0..1.0.
487 // With K = 2:
488 //   severity 0%   -> 100% straight speed
489 //   severity 25%  -> ~82%
490 //   severity 50%  -> ~71%
491 //   severity 75%  -> ~63%
492 //   severity 100% -> ~58%
493 // In physical race mode with 1050 RPM straight speed this is
494 // roughly:
495 //   0%   -> 1050 RPM ~= 2.36 m/s
496 //   25%  -> 857 RPM  ~= 1.93 m/s
497 //   50%  -> 742 RPM  ~= 1.67 m/s
498 //   75%  -> 664 RPM  ~= 1.50 m/s
499 //   100% -> 606 RPM  ~= 1.36 m/s
```

Integrated autonomous controller

ORIGINAL LINES 500-538

```
500 // The Pi's own speed recommendation can request an EVEN LOWER
501 // speed. STM32 always chooses the safer/lower of the two.
502 // =====
503 const float CORNER_CURVATURE_GAIN = 2.0f;
504 int visionSteerPct = 0;
505 int visionSpeedPct = 100;
506 int visionConfidencePct = 0;
507 // 0..100 confidence that the camera sees a bridge/ramp ahead.
508 int visionBridgeConfidencePct = 0;
509 // 0 = straight road ahead, 100 = strongest expected corner.
510 // This is look-ahead geometry, not merely current steering angle.
511 int visionCornerSeverityPct = 0;
512 bool visionPacketValid = false;
513 unsigned long lastVisionPacketTime = 0;
514 char visionRxBuffer[48];
515 uint8_t visionRxIndex = 0;
516 // Pi UART diagnostics - visible in the NORMAL Serial Monitor.
517 unsigned long piRxByteCount = 0;
518 unsigned long piGoodPacketCount = 0;
519 unsigned long piBadPacketCount = 0;
520 char piLastRawPacket[48] = "(none)";
521 // =====
522 // TOF / TTC
523 // =====
524 // 20 Hz ToF/TTC update.
525 // A 500 ms interval is too slow for a racing robot.
526 const unsigned long TOF_INTERVAL_MS = 50;
527 const float MIN_CLOSING_SPEED_MM_S = 50.0f;
528 const float TTC_CRITICAL_S = 0.50f;
529 const float TTC_AVOID_S = 1.00f;
530 const float TTC_CAUTION_S = 2.00f;
531 const uint16_t CRITICAL_RANGE_MM = 180;
532 const uint16_t AVOID_RANGE_MM = 400;
533 const uint16_t CAUTION_RANGE_MM = 800;
534 const uint16_t SIDE_TOLERANCE_MM = 100;
535 // =====
536 // INA219 CURRENT / POWER SENSOR
537 // =====
538 // Wokwi behavioural model:
```

Integrated autonomous controller

ORIGINAL LINES 539-577

```
539 // Address 0x40
540 // Register 0x02 = bus voltage
541 // Register 0x04 = current
542 // IMPORTANT:
543 // These current thresholds are SIMULATION STARTING VALUES.
544 // Tune them against the real motor stall-current measurements
545 // before using them on the physical robot.
546 // =====
547 #define INA219_ADDR          0x40
548 #define INA219_REG_BUS_VOLTAGE 0x02
549 #define INA219_REG_CURRENT    0x04
550 const unsigned long INA_INTERVAL_MS = 100;
551 // Wokwi keeps the already-tested slider thresholds.
552 // Physical starting values are intentionally conservative and
553 // must be calibrated on the assembled car. The encoder-speed
554 // condition + 1 s confirmation + manual reset latch remain active.
555 const float HIGH_CURRENT_MA = RUNNING_IN_WOKWI ? 1200.0f : 500.0f;
556 const float STALL_CURRENT_MA = RUNNING_IN_WOKWI ? 1800.0f : 650.0f;
557 // Reference only. Confirmed stall uses MAIN OFF -> ON reset.
558 const float STALL_RESET_CURRENT_MA = RUNNING_IN_WOKWI ? 800.0f : 400.0f;
559 const float STALL_RPM_THRESHOLD = RUNNING_IN_WOKWI ? 5.0f : 80.0f;
560 const float STALL_COMMAND_RPM_THRESHOLD = RUNNING_IN_WOKWI ? 10.0f : 200.0f;
561 const unsigned long STALL_CONFIRM_MS = 1000;
562 // =====
563 // ENCODERS
564 // =====
565 const float CPR = 280.0f;
566 // RPM sanity range.
567 // Wokwi: 42.86 RPM model -> 60 RPM sanity ceiling.
568 // Physical: 1500 RPM no-load motor -> 1800 RPM diagnostic ceiling.
569 const float MAX_VALID_RPM = RUNNING_IN_WOKWI ? 60.0f : 1800.0f;
570 const int8_t quadratureTable[16] = {
571     0, +1, -1,  0, -1,  0,  0, +1, +1,  0,  0, -1, 0, -1, +1,  0
572 };
573 // =====
574 // PID
575 // =====
576 // Wokwi gains remain exactly as tested.
577 // Physical gains are safe STARTING VALUES only and require
```


Integrated autonomous controller

ORIGINAL LINES 578-616

```
578 // real encoder/PWM tuning before competition-speed operation.
579 float Kp = RUNNING_IN_WOKWI ? 2.0f : 0.25f;
580 float Ki = RUNNING_IN_WOKWI ? 0.60f : 0.08f;
581 float Kd = RUNNING_IN_WOKWI ? 0.05f : 0.01f;
582 // 20 Hz wheel-speed PID for the high-speed Wokwi build.
583 // IR road-edge control remains faster at 100 Hz.
584 const unsigned long PID_INTERVAL_MS = 50;
585 // =====
586 // PID STABILITY / OVERSPEED CONTROL
587 // =====
588 // Wokwi N20 model calibration:
589 //   255 PWM -> about 42.86 RPM
590 // We therefore use a speed-to-PWM feed-forward term.
591 // PID only trims the remaining error.
592 // Physical robot:
593 // Measure the real PWM-vs-RPM relationship and retune.
594 // =====
595 const float WOKWI_MOTOR_MAX_RPM = 42.86f;
596 const float PHYSICAL_MOTOR_NO_LOAD_RPM = 1500.0f;
597 // Physical feed-forward is only a first approximation.
598 // Replace it with measured PWM-vs-loaded-RPM data.
599 const float PWM_FEEDFORWARD_PER_RPM = RUNNING_IN_WOKWI ? ( 255.0f / WOKWI_MOTOR_MAX_RPM ) : ( 255.0f / PHYSICAL_MOTOR_NO_LOAD_RPM );
600 // Integral anti-windup bound.
601 const float PID_INTEGRAL_LIMIT = RUNNING_IN_WOKWI ? 15.0f : 400.0f;
602 // Smooth encoder RPM samples before PID.
603 const float RPM_FILTER_ALPHA = 0.35f;
604 // Bleed positive integral if actual RPM exceeds setpoint.
605 const float OVERSPEED_MARGIN_RPM = RUNNING_IN_WOKWI ? 1.0f : 25.0f;
606 const float OVERSPEED_INTEGRAL_BLEED = 0.50f;
607 // =====
608 // SPEED RAMP / SLEW-RATE LIMITER
609 // =====
610 // Navigation can request a new wheel speed immediately,
611 // but the PID follows a ramped setpoint instead of jumping
612 // straight to the requested value.
613 // This reduces launch wheel-slip, current spikes, false
614 // stall detection, and violent weight transfer.
615 // Wokwi starting values:
616 //   acceleration = 30 RPM/s
```

Integrated autonomous controller

ORIGINAL LINES 617-654

```
617 // deceleration = 120 RPM/s
618 // At the 50 ms PID interval this is about:
619 // +1.5 RPM per PID step while accelerating
620 // -6.0 RPM per PID step while decelerating
621 // Deceleration is deliberately faster so the robot can
622 // react quickly to corners and boundaries.
623 // IMPORTANT: these are simulation values. The physical
624 // 1500 RPM N20 robot must be retuned using measured grip,
625 // current draw, mass, bridge slope and wheel-slip.
626 // =====
627 const float ACCEL_RAMP_RPM_PER_SEC = RUNNING_IN_WOKWI ? 30.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 300.0f : 450.0f );
628 const float DECEL_RAMP_RPM_PER_SEC = RUNNING_IN_WOKWI ? 120.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 450.0f : 700.0f );
629 // =====
630 // SOFTWARE PWM
631 // =====
632 const unsigned long PWM_PERIOD_US = 10000;
633 // =====
634 // AVOIDANCE SPEED PROFILE
635 // =====
636 const float AVOID_SLOW_RPM = RUNNING_IN_WOKWI ? 14.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 220.0f : 400.0f );
637 const float AVOID_FAST_RPM = RUNNING_IN_WOKWI ? 34.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 540.0f : 900.0f );
638 const float CRITICAL_AVOID_SLOW_RPM = RUNNING_IN_WOKWI ? 8.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 120.0f : 180.0f );
639 const float CRITICAL_AVOID_FAST_RPM = RUNNING_IN_WOKWI ? 38.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 600.0f : 1050.0f );
640 // =====
641 // NAVIGATION STATES
642 // =====
643 enum NavigationState {
644     STATE_EMERGENCY_STOP, STATE_VISION_DRIVE, STATE_VISION_FALLBACK, STATE_ROAD_DRIVE, STATE_BOUNDARY_CORRECT_LEFT, STATE_BOUNDARY_CORRE
CT_RIGHT,
645     STATE_BOUNDARY_CAUTION, STATE_BOUNDARY_ESCAPE, STATE_CAUTION, STATE_AVOID_LEFT, STATE_AVOID_RIGHT, STATE_CRITICAL_AVOID_LEFT,
646     STATE_CRITICAL_AVOID_RIGHT, STATE_CRITICAL_STOP,
647     // Obstacle requires an avoidance turn, but the road
648     // boundary blocks that escape direction.
649     STATE_NO_SAFE_PATH, STATE_MOTOR_STALL, STATE_SKID_CAUTION, STATE_SENSOR_FAULT,
650     // Appended so all previously validated state numbers stay unchanged.
651     STATE_SYSTEM_OFF, STATE_BRIDGE_APPROACH, STATE_BRIDGE_CLIMB, STATE_BRIDGE_CREST, STATE_BRIDGE_DESCENT, STATE_BRIDGE_EXIT
652 };
653 // Safe boot: the race controller always starts OFF.
654 // It cannot drive until the MAIN switch has been observed OFF
```

Integrated autonomous controller

ORIGINAL LINES 655-693

```
655 // and is then deliberately switched ON.
656 NavigationState currentState = STATE_SYSTEM_OFF;
657 NavigationState previousState = STATE_SYSTEM_OFF;
658 // Counts impossible/invalid state values detected at runtime.
659 // Any invalid state immediately falls back to CRITICAL STOP.
660 unsigned long invalidStateRecoveryCount = 0;
661 bool emergencyStopActive = false;
662 // =====
663 // DEBOUNCED MAIN SWITCH + RACE-START INTERLOCK
664 // =====
665 // mainSwitchRawOn:
666 //   immediate electrical reading from PA15.
667 // mainSwitchOn:
668 //   stable/debounced switch state used by the controller.
669 // The robot ALWAYS boots with the race interlock locked.
670 // It must observe a stable MAIN OFF state, and only a later
671 // stable OFF -> ON transition is allowed to start the race.
672 // This also prevents one physical switch movement from being
673 // interpreted as several starts because of contact bounce.
674 // =====
675 bool mainSwitchRawOn = false;
676 bool mainSwitchOn = false;
677 bool mainSwitchSeenOffSinceBoot = false;
678 bool raceStartInterlockCleared = false;
679 unsigned long mainSwitchLastRawChangeMs = 0;
680 // 60 ms is comfortably above normal mechanical switch bounce
681 // and is negligible compared with race setup time.
682 const unsigned long MAIN_SWITCH_DEBOUNCE_MS = 60;
683 // =====
684 // ROAD / BOUNDARY DATA
685 // =====
686 // boundarySteerError:
687 //   positive -> steer RIGHT
688 //   negative -> steer LEFT
689 // leftBoundarySeverity/rightBoundarySeverity:
690 //   0 = no boundary
691 //   1 = outer sensor
692 //   2 = next sensor
693 //   3 = inner sensor
```

Integrated autonomous controller

ORIGINAL LINES 694-732

```
694 // 4 = centre-adjacent sensor
695 // =====
696 float boundarySteerError = 0.0f;
697 float lastBoundarySteerError = 0.0f;
698 float leftBoundarySeverity = 0.0f;
699 float rightBoundarySeverity = 0.0f;
700 float boundarySeverity = 0.0f;
701 bool boundaryDetected = false;
702 bool centreBoundaryDetected = false;
703 bool bothSidesBoundaryDetected = false;
704 int activeBoundarySensors = 0;
705 // =====
706 // TOF DATA
707 // =====
708 uint16_t leftDistance = 2000;
709 uint16_t rightDistance = 2000;
710 uint16_t previousLeftDistance = 2000;
711 uint16_t previousRightDistance = 2000;
712 float leftClosingSpeed = 0.0f;
713 float rightClosingSpeed = 0.0f;
714 float leftTTC = -1.0f;
715 float rightTTC = -1.0f;
716 bool tofValid = false;
717 bool previousToFValid = false;
718 unsigned long previousToFTime = 0;
719 // =====
720 // INA219 DATA
721 // =====
722 float busVoltageV = 0.0f;
723 float motorCurrentMA = 0.0f;
724 bool inaValid = false;
725 bool motorStallDetected = false;
726 bool motorStallLatched = false;
727 // Bit 0 = left wheel, bit 1 = right wheel.
728 // These let us identify a one-wheel stall instead of averaging
729 // both wheel speeds together.
730 uint8_t stallCandidateMask = 0;
731 uint8_t stallLatchedSideMask = 0;
732 unsigned long previousINATime = 0;
```

Integrated autonomous controller

ORIGINAL LINES 733-771

```
733 unsigned long stallConditionStart = 0;
734 // =====
735 // IMU
736 // =====
737 float gyroZ = 0.0f;
738 // 0 = flat, positive = climbing, negative = descending.
739 float pitchDeg = 0.0f;
740 bool imuValid = false;
741 bool possibleSkid = false;
742 enum BridgePhase {
743     BRIDGE_NONE, BRIDGE_APPROACH, BRIDGE_CLIMB, BRIDGE_CREST, BRIDGE_DESCENT, BRIDGE_EXIT
744 };
745 BridgePhase bridgePhase = BRIDGE_NONE;
746 float maximumBridgeClimbPitchDeg = 0.0f;
747 unsigned long bridgeExitStartTime = 0;
748 // =====
749 // ENCODER STATE
750 // =====
751 long leftEncoderCount = 0;
752 long rightEncoderCount = 0;
753 long previousLeftPIDCount = 0;
754 long previousRightPIDCount = 0;
755 uint8_t previousLeftEncoderState = 0;
756 uint8_t previousRightEncoderState = 0;
757 // =====
758 // RPM
759 // =====
760 // Filtered encoder RPM used by the controller.
761 float leftRPM = 0.0f;
762 float rightRPM = 0.0f;
763 // Latest raw encoder RPM samples.
764 float leftRawRPM = 0.0f;
765 float rightRawRPM = 0.0f;
766 bool leftRPMFilterInitialised = false;
767 bool rightRPMFilterInitialised = false;
768 // True when the most recent encoder RPM sample was finite
769 // and inside the physically reasonable simulation range.
770 bool leftRPMValid = true;
771 bool rightRPMValid = true;
```

Integrated autonomous controller

ORIGINAL LINES 772-810

```
772 // Navigation-requested wheel speeds.
773 float leftTargetRPM = NORMAL_BASE_RPM;
774 float rightTargetRPM = NORMAL_BASE_RPM;
775 // PID setpoints after acceleration/deceleration limiting.
776 // Start at zero for a controlled soft launch.
777 float leftRampedTargetRPM = 0.0f;
778 float rightRampedTargetRPM = 0.0f;
779 // =====
780 // PID STATE
781 // =====
782 float leftIntegral = 0.0f;
783 float rightIntegral = 0.0f;
784 float leftPreviousError = 0.0f;
785 float rightPreviousError = 0.0f;
786 unsigned long previousPIDTime = 0;
787 // =====
788 // PWM STATE
789 // =====
790 // Start with zero motor drive. PID increases PWM only
791 // as the ramped target rises.
792 int leftPWM = 0;
793 int rightPWM = 0;
794 bool leftPWMState = false;
795 bool rightPWMState = false;
796 unsigned long pwmPeriodStart = 0;
797 // =====
798 // SERIAL TIMING
799 // =====
800 const unsigned long PRINT_INTERVAL_MS = 1500;
801 unsigned long previousPrintTime = 0;
802 // =====
803 // ENCODERS
804 // =====
805 void readLeftEncoder() {
806     uint8_t a = digitalRead(LEFT_ENC_A);
807     uint8_t b = digitalRead(LEFT_ENC_B);
808     uint8_t current = (a << 1) | b;
809     if (current == previousLeftEncoderState) return;
810     uint8_t index = (previousLeftEncoderState << 2) | current;
```

Integrated autonomous controller

ORIGINAL LINES 811-849

```
811     leftEncoderCount += quadratureTable[index];
812     previousLeftEncoderState = current;
813 }
814 void readRightEncoder() {
815     uint8_t a = digitalRead(RIGHT_ENC_A);
816     uint8_t b = digitalRead(RIGHT_ENC_B);
817     uint8_t current = (a << 1) | b;
818     if (current == previousRightEncoderState) return;
819     uint8_t index = (previousRightEncoderState << 2) | current;
820     rightEncoderCount += quadratureTable[index];
821     previousRightEncoderState = current;
822 }
823 // =====
824 // SOFTWARE PWM
825 // =====
826 void updateSoftwarePWM() {
827     unsigned long now = micros();
828     if ( now - pwmPeriodStart >= PWM_PERIOD_US ) {
829         pwmPeriodStart = now;
830     }
831     unsigned long position = now - pwmPeriodStart;
832     // LEFT
833     unsigned long leftHighTime = ((unsigned long)leftPWM * PWM_PERIOD_US) / 255UL;
834     bool desiredLeft;
835     if (leftPWM <= 0) desiredLeft = false;
836     else if (leftPWM >= 255) desiredLeft = true;
837     else desiredLeft = position < leftHighTime;
838     if (desiredLeft != leftPWMState) {
839         leftPWMState = desiredLeft;
840         digitalWrite( PWMA, leftPWMState ? HIGH : LOW );
841     }
842     // RIGHT
843     unsigned long rightHighTime = ((unsigned long)rightPWM * PWM_PERIOD_US) / 255UL;
844     bool desiredRight;
845     if (rightPWM <= 0) desiredRight = false;
846     else if (rightPWM >= 255) desiredRight = true;
847     else desiredRight = position < rightHighTime;
848     if (desiredRight != rightPWMState) {
849         rightPWMState = desiredRight;
```

Integrated autonomous controller

ORIGINAL LINES 850-888

```
850     digitalWrite( PWMB, rightPWMState ? HIGH : LOW );
851 }
852 }
853 void serviceMotors() {
854     updateSoftwarePWM();
855     readLeftEncoder();
856     readRightEncoder();
857 }
858 // =====
859 // MOTOR DIRECTION
860 // =====
861 void motorsForward() {
862     // Direction commands are prepared here. The motor
863     // driver is enabled only while the E-Stop is released.
864     digitalWrite( STBY, ( emergencyStopActive || !mainSwitchOn || !raceStartInterlockCleared ) ? LOW : HIGH );
865     digitalWrite(AIN1, HIGH);
866     digitalWrite(AIN2, LOW);
867     digitalWrite(BIN1, HIGH);
868     digitalWrite(BIN2, LOW);
869 }
870 // =====
871 // EMERGENCY STOP
872 // =====
873 void readEmergencyStop() {
874     emergencyStopActive = ( digitalRead(ESTOP_PIN) == LOW );
875 }
876 void applyEmergencyStopImmediately() {
877     // Do not wait for the 250 ms PID update.
878     currentState = STATE_EMERGENCY_STOP;
879     leftTargetRPM = 0.0f;
880     rightTargetRPM = 0.0f;
881     leftRampedTargetRPM = 0.0f;
882     rightRampedTargetRPM = 0.0f;
883     leftPWM = 0;
884     rightPWM = 0;
885     leftPWMState = false;
886     rightPWMState = false;
887     digitalWrite(PWMA, LOW);
888     digitalWrite(PWMB, LOW);
```


Integrated autonomous controller

ORIGINAL LINES 889-927

```
889 // Hardware-driver shutdown in the Wokwi controller.
890 digitalWrite(STBY, LOW);
891 }
892 // =====
893 // MAIN ON/OFF SWITCH
894 // =====
895 void resetDriveMemoryForRaceStart() {
896 // Start both motor channels from exactly the same neutral state.
897 leftTargetRPM = 0.0f;
898 rightTargetRPM = 0.0f;
899 leftRampedTargetRPM = 0.0f;
900 rightRampedTargetRPM = 0.0f;
901 leftIntegral = 0.0f;
902 rightIntegral = 0.0f;
903 leftPreviousError = 0.0f;
904 rightPreviousError = 0.0f;
905 leftPWM = 0;
906 rightPWM = 0;
907 leftPWMState = false;
908 rightPWMState = false;
909 // Neutral navigation memory.
910 boundarySteerError = 0.0f;
911 lastBoundarySteerError = 0.0f;
912 // Neutral internal vision placeholder.
913 visionSteerPct = 0;
914 visionSpeedPct = 100;
915 visionConfidencePct = 95;
916 visionBridgeConfidencePct = 0;
917 visionCornerSeverityPct = 0;
918 // No scripted test is allowed to carry into the race.
919 bridgePhase = BRIDGE_NONE;
920 wokwiBridgeTestStarted = false;
921 wokwiCornerTestStarted = false;
922 // PID timing starts fresh so the first ramp step is symmetrical.
923 previousPIDTime = millis();
924 // Motor driver remains disabled until the main loop reaches
925 // the normal enabled section.
926 digitalWrite(PWMA, LOW);
927 digitalWrite(PWMB, LOW);
```

Integrated autonomous controller

ORIGINAL LINES 928-966

```
928     digitalWrite(STBY, LOW);
929 }
930 void initialiseMainSwitch() {
931     // INPUT_PULLUP:
932     // LOW  = ON
933     // HIGH = OFF
934     mainSwitchRawOn = ( digitalRead(MAIN_SWITCH_PIN) == LOW );
935     mainSwitchOn = mainSwitchRawOn;
936     mainSwitchLastRawChangeMs = millis();
937     // IMPORTANT:
938     // Never start a race merely because the switch happened to be
939     // ON when the MCU/Wokwi simulation booted.
940     raceStartInterlockCleared = false;
941     if (!mainSwitchOn) {
942         mainSwitchSeenOffSinceBoot = true;
943     }
944     else {
945         mainSwitchSeenOffSinceBoot = false;
946     }
947     // Safe startup outputs.
948     applySystemOffImmediately();
949 }
950 void readMainSwitch() {
951     bool rawOn = ( digitalRead(MAIN_SWITCH_PIN) == LOW );
952     // A raw transition only starts/restarts the debounce timer.
953     // It does NOT immediately change the controller state.
954     if (rawOn != mainSwitchRawOn) {
955         mainSwitchRawOn = rawOn;
956         mainSwitchLastRawChangeMs = millis();
957     }
958     // Wait until the raw state has remained unchanged long enough.
959     if ( (millis() - mainSwitchLastRawChangeMs) < MAIN_SWITCH_DEBOUNCE_MS ) {
960         return;
961     }
962     // No new stable switch transition.
963     if (mainSwitchRawOn == mainSwitchOn) {
964         return;
965     }
966     // Accept exactly one stable transition.
```

Integrated autonomous controller

ORIGINAL LINES 967-1,005

```
967   bool oldStableOn = mainSwitchOn;
968   mainSwitchOn = mainSwitchRawOn;
969   // =====
970   // STABLE ON -> OFF
971   // =====
972   if ( oldStableOn && !mainSwitchOn ) {
973       // The robot is now deliberately OFF.
974       raceStartInterlockCleared = false;
975       mainSwitchSeenOffSinceBoot = true;
976       // A confirmed motor stall may only be cleared by this
977       // deliberate MAIN OFF action.
978       resetMotorStallLatchWhileSystemOff();
979       // Immediately make both motor channels identical and safe.
980       applySystemOffImmediately();
981       DebugSerial.println( "[POWER] MAIN SWITCH -> OFF : SYSTEM SAFE" );
982       return;
983   }
984   // =====
985   // FIRST/STABLE OFF OBSERVATION
986   // =====
987   if (!mainSwitchOn) {
988       mainSwitchSeenOffSinceBoot = true;
989       raceStartInterlockCleared = false;
990       applySystemOffImmediately();
991       return;
992   }
993   // =====
994   // STABLE OFF -> ON = ONE RACE START
995   // =====
996   if ( !oldStableOn && mainSwitchOn && mainSwitchSeenOffSinceBoot && !raceStartInterlockCleared ) {
997       resetDriveMemoryForRaceStart();
998       raceStartInterlockCleared = true;
999       DebugSerial.println( "[START] MAIN OFF -> ON : RACE START ENABLED" );
1000       return;
1001   }
1002   // Any other situation remains locked for safety.
1003   raceStartInterlockCleared = false;
1004   }
1005   void applySystemOffImmediately() {
```

Integrated autonomous controller

ORIGINAL LINES 1,006-1,044

```
1006     currentState = STATE_SYSTEM_OFF;
1007     leftTargetRPM = 0.0f;
1008     rightTargetRPM = 0.0f;
1009     leftRampedTargetRPM = 0.0f;
1010     rightRampedTargetRPM = 0.0f;
1011     leftPWM = 0;
1012     rightPWM = 0;
1013     leftPWMState = false;
1014     rightPWMState = false;
1015     leftIntegral = 0.0f;
1016     rightIntegral = 0.0f;
1017     digitalWrite(PWMA, LOW);
1018     digitalWrite(PWMB, LOW);
1019     digitalWrite(STBY, LOW);
1020 }
1021 // =====
1022 // WOKWI AUTONOMOUS BRIDGE SCENARIO
1023 // =====
1024 void updateWokwiBridgeScenario() {
1025     if (!USE_WOKWI_INTERNAL_IMU_SIM) {
1026         return;
1027     }
1028     if (!WOKWI_AUTO_BRIDGE_TEST) {
1029         wokwiSimulatedPitchDeg = WOKWI_MANUAL_PITCH_DEG;
1030         wokwiSimulatedBridgeConfidencePct = WOKWI_MANUAL_BRIDGE_CONFIDENCE_PCT;
1031         return;
1032     }
1033     if (!wokwiBridgeTestStarted) {
1034         wokwiBridgeTestStarted = true;
1035         wokwiBridgeTestStartTime = millis();
1036     }
1037     unsigned long elapsed = millis() - wokwiBridgeTestStartTime;
1038     if (elapsed < BRIDGE_TEST_FLAT_MS) {
1039         wokwiSimulatedPitchDeg = 0.0f;
1040         wokwiSimulatedBridgeConfidencePct = 0;
1041         return;
1042     }
1043     if (elapsed < BRIDGE_TEST_APPROACH_MS) {
1044         wokwiSimulatedPitchDeg = 0.0f;
```

Integrated autonomous controller

ORIGINAL LINES 1,045-1,083

```
1045     wokwiSimulatedBridgeConfidencePct = 90;
1046     return;
1047 }
1048 if (elapsed < BRIDGE_TEST_CLIMB_MS) {
1049     wokwiSimulatedPitchDeg = 4.5f;
1050     wokwiSimulatedBridgeConfidencePct = 95;
1051     return;
1052 }
1053 if (elapsed < BRIDGE_TEST_CREST_MS) {
1054     wokwiSimulatedPitchDeg = 1.0f;
1055     wokwiSimulatedBridgeConfidencePct = 95;
1056     return;
1057 }
1058 if (elapsed < BRIDGE_TEST_DESCENT_MS) {
1059     wokwiSimulatedPitchDeg = -4.5f;
1060     wokwiSimulatedBridgeConfidencePct = 90;
1061     return;
1062 }
1063 if (elapsed < BRIDGE_TEST_EXIT_MS) {
1064     wokwiSimulatedPitchDeg = 0.0f;
1065     wokwiSimulatedBridgeConfidencePct = 20;
1066     return;
1067 }
1068 wokwiSimulatedPitchDeg = 0.0f;
1069 wokwiSimulatedBridgeConfidencePct = 0;
1070 }
1071 // =====
1072 // RASPBERRY PI VISION UART
1073 // =====
1074 bool visionIsOnline() {
1075     if (!visionPacketValid) return false;
1076     return ( millis() - lastVisionPacketTime ) <= VISION_TIMEOUT_MS;
1077 }
1078 bool visionIsTrusted() {
1079     return visionIsOnline() && visionConfidencePct >= VISION_MIN_CONFIDENCE_PCT;
1080 }
1081 const char *cornerClassName() {
1082     if (visionCornerSeverityPct < 15) {
1083         return "STRAIGHT";
```

Integrated autonomous controller

ORIGINAL LINES 1,084-1,122

```
1084     }
1085     if (visionCornerSeverityPct < 35) {
1086         return "GENTLE";
1087     }
1088     if (visionCornerSeverityPct < 60) {
1089         return "MEDIUM";
1090     }
1091     if (visionCornerSeverityPct < 85) {
1092         return "SHARP";
1093     }
1094     return "VERY SHARP";
1095 }
1096 float getVisionCornerSpeedCapRPM() {
1097     if (!visionIsTrusted()) {
1098         return VISION_FALLBACK_RPM;
1099     }
1100     float severity = constrain( (float)visionCornerSeverityPct / 100.0f, 0.0f, 1.0f );
1101     // F = m*v^2/R means the speed limit should fall
1102     // non-linearly as curvature demand rises.
1103     float speedFactor = 1.0f / sqrtf( 1.0f + CORNER_CURVATURE_GAIN * severity );
1104     float capRPM = NORMAL_BASE_RPM * speedFactor;
1105     return constrain( capRPM, VISION_MIN_BASE_RPM, NORMAL_BASE_RPM );
1106 }
1107 float getVisionBaseRPM() {
1108     if (!visionIsTrusted()) {
1109         return VISION_FALLBACK_RPM;
1110     }
1111     int safeSpeedPct = constrain( visionSpeedPct, VISION_MIN_SPEED_PCT, VISION_MAX_SPEED_PCT );
1112     float cameraRequestedRPM = NORMAL_BASE_RPM * ( (float)safeSpeedPct / 100.0f );
1113     float cornerSpeedCapRPM = getVisionCornerSpeedCapRPM();
1114     float saferRPM = min( cameraRequestedRPM, cornerSpeedCapRPM );
1115     return constrain( saferRPM, VISION_MIN_BASE_RPM, NORMAL_BASE_RPM );
1116 }
1117 bool parseVisionPacket( char *packet ) {
1118     int steer = 0;
1119     int speed = 0;
1120     int confidence = 0;
1121     int bridgeConfidence = 0;
1122     int cornerSeverity = 0;
```

Integrated autonomous controller

ORIGINAL LINES 1,123-1,161

```
1123 // Backward compatible packet formats:
1124 // V,steer,speed,confidence
1125 // V,steer,speed,confidence,bridgeConfidence
1126 // V,steer,speed,confidence,bridgeConfidence,cornerSeverity
1127 int parsed = sscanf( packet, "V,%d,%d,%d,%d,%d", &steer, &speed, &confidence, &bridgeConfidence, &cornerSeverity );
1128 if ( parsed != 3 && parsed != 4 && parsed != 5 ) {
1129     return false;
1130 }
1131 visionSteerPct = constrain( steer, -100, 100 );
1132 visionSpeedPct = constrain( speed, VISION_MIN_SPEED_PCT, VISION_MAX_SPEED_PCT );
1133 visionConfidencePct = constrain( confidence, 0, 100 );
1134 visionBridgeConfidencePct = constrain( parsed >= 4 ? bridgeConfidence : 0, 0, 100 );
1135 visionCornerSeverityPct = constrain( parsed >= 5 ? cornerSeverity : 0, 0, 100 );
1136 visionPacketValid = true;
1137 lastVisionPacketTime = millis();
1138 return true;
1139 }
1140 void getWokwiCornerTestCommand( int &steerPct, int &speedPct, int &cornerSeverityPct ) {
1141     steerPct = WOKWI_VISION_STEER_PCT;
1142     speedPct = WOKWI_VISION_SPEED_PCT;
1143     cornerSeverityPct = WOKWI_VISION_CORNER_SEVERITY_PCT;
1144     if ( !WOKWI_AUTO_CORNER_SPEED_TEST ) {
1145         return;
1146     }
1147     if ( !wokwiCornerTestStarted ) {
1148         wokwiCornerTestStarted = true;
1149         wokwiCornerTestStartTime = millis();
1150     }
1151     unsigned long elapsed = millis() - wokwiCornerTestStartTime;
1152     // 0-3 s: STRAIGHT
1153     if ( elapsed < CORNER_TEST_GENTLE_MS ) {
1154         steerPct = 0;
1155         speedPct = 100;
1156         cornerSeverityPct = 0;
1157         return;
1158     }
1159     // 3-6 s: GENTLE RIGHT
1160     if ( elapsed < CORNER_TEST_MEDIUM_MS ) {
1161         steerPct = 20;
```

Integrated autonomous controller

ORIGINAL LINES 1,162-1,200

```
1162     speedPct = 100;
1163     cornerSeverityPct = 25;
1164     return;
1165 }
1166 // 6-9 s: MEDIUM LEFT
1167 if (elapsed < CORNER_TEST_SHARP_MS) {
1168     steerPct = -40;
1169     speedPct = 100;
1170     cornerSeverityPct = 50;
1171     return;
1172 }
1173 // 9-12 s: SHARP RIGHT
1174 if (elapsed < CORNER_TEST_VERY_SHARP_MS) {
1175     steerPct = 60;
1176     speedPct = 100;
1177     cornerSeverityPct = 75;
1178     return;
1179 }
1180 // 12-15 s: VERY SHARP LEFT
1181 if (elapsed < CORNER_TEST_END_MS) {
1182     steerPct = -75;
1183     speedPct = 100;
1184     cornerSeverityPct = 100;
1185     return;
1186 }
1187 // >15 s: STRAIGHT AGAIN
1188 steerPct = 0;
1189 speedPct = 100;
1190 cornerSeverityPct = 0;
1191 }
1192 void updateVisionSerial() {
1193     // =====
1194     // WOKWI INTERNAL VISION MODE
1195     // =====
1196     // This bypasses the custom Pi UART chip entirely.
1197     // It tests the SAME downstream STM32 navigation logic
1198     // that the real Pi packets will use later.
1199     // =====
1200     if (USE_WOKWI_INTERNAL_VISION_SIM) {
```


Integrated autonomous controller

ORIGINAL LINES 1,201-1,239

```
1201 unsigned long now = millis();
1202 if ( now - previousWokwiVisionTime < WOKWI_VISION_INTERVAL_MS ) {
1203     return;
1204 }
1205 previousWokwiVisionTime = now;
1206 if (!WOKWI_VISION_ONLINE) {
1207     // Send nothing. The normal 250 ms timeout will
1208     // naturally change VISION STATUS to OFFLINE / STALE.
1209     return;
1210 }
1211 int simulatedSteerPct = 0;
1212 int simulatedSpeedPct = 100;
1213 int simulatedCornerSeverityPct = 0;
1214 getWokwiCornerTestCommand( simulatedSteerPct, simulatedSpeedPct, simulatedCornerSeverityPct );
1215 visionSteerPct = constrain( simulatedSteerPct, -100, 100 );
1216 visionSpeedPct = constrain( simulatedSpeedPct, VISION_MIN_SPEED_PCT, VISION_MAX_SPEED_PCT );
1217 visionCornerSeverityPct = constrain( simulatedCornerSeverityPct, 0, 100 );
1218 visionConfidencePct = constrain( WOKWI_VISION_CONFIDENCE_PCT, 0, 100 );
1219 visionBridgeConfidencePct = constrain( wokwiSimulatedBridgeConfidencePct, 0, 100 );
1220 visionPacketValid = true;
1221 lastVisionPacketTime = now;
1222 // Keep the existing diagnostics useful.
1223 snprintf( piLastRawPacket, sizeof(piLastRawPacket), "SIM,V,%d,%d,%d,%d,%d", visionSteerPct, visionSpeedPct, visionConfidencePct,
1224     visionBridgeConfidencePct, visionCornerSeverityPct );
1225 piGoodPacketCount++;
1226 return;
1227 }
1228 // =====
1229 // PHYSICAL RASPBERRY PI UART MODE
1230 // =====
1231 // Set USE_WOKWI_INTERNAL_VISION_SIM = false to use this
1232 // path. The real Pi sends:
1233 //   V,<steerPct>,<speedPct>,<confidencePct>,<bridgeConfidencePct>,<cornerSeverityPct>\n
1234 // Older 4-field and 5-field packets remain accepted.
1235 // =====
1236 while (PiSerial.available() > 0) {
1237     char c = (char)PiSerial.read();
1238     piRxByteCount++;
1239     if ( c == '\n' || c == '\r' ) {
```

Integrated autonomous controller

ORIGINAL LINES 1,240-1,278

```
1240     if (visionRxIndex > 0) {
1241         visionRxBuffer[ visionRxIndex ] = '\0';
1242         strncpy( piLastRawPacket, visionRxBuffer, sizeof(piLastRawPacket) - 1 );
1243         piLastRawPacket[ sizeof(piLastRawPacket) - 1 ] = '\0';
1244         if ( parseVisionPacket( visionRxBuffer ) ) {
1245             piGoodPacketCount++;
1246         }
1247         else {
1248             piBadPacketCount++;
1249         }
1250         visionRxIndex = 0;
1251     }
1252     continue;
1253 }
1254 if ( visionRxIndex < sizeof(visionRxBuffer) - 1 ) {
1255     visionRxBuffer[ visionRxIndex++ ] = c;
1256 }
1257 else {
1258     visionRxIndex = 0;
1259     piBadPacketCount++;
1260 }
1261 }
1262 }
1263 // =====
1264 // CAMERA PREDICTIVE MOTOR TARGETS
1265 // =====
1266 void applyVisionGuidanceAtBase( float requestedBaseRPM ) {
1267     float baseRPM = min( getVisionBaseRPM(), requestedBaseRPM );
1268     float normalizedSteer = (float)visionSteerPct / 100.0f;
1269     float correctionRPM = VISION_MAX_STEER_RPM * normalizedSteer;
1270     // Positive steering means RIGHT:
1271     // left wheel faster, right wheel slower.
1272     leftTargetRPM = baseRPM + correctionRPM;
1273     rightTargetRPM = baseRPM - correctionRPM;
1274     leftTargetRPM = constrain( leftTargetRPM, MIN_TARGET_RPM, MAX_TARGET_RPM );
1275     rightTargetRPM = constrain( rightTargetRPM, MIN_TARGET_RPM, MAX_TARGET_RPM );
1276 }
1277 void applyVisionGuidance() {
1278     applyVisionGuidanceAtBase( NORMAL_BASE_RPM );
```

Integrated autonomous controller

ORIGINAL LINES 1,279-1,317

```
1279 }
1280 // =====
1281 // ROAD-BOUNDARY SENSORS
1282 // =====
1283 void readBoundarySensors() {
1284     for (int i = 0; i < 8; i++) {
1285         sensor[i] = digitalRead(IR_PINS[i]);
1286     }
1287 }
1288 void analyseRoadBoundaries() {
1289     float weightedSum = 0.0f;
1290     activeBoundarySensors = 0;
1291     leftBoundarySeverity = 0.0f;
1292     rightBoundarySeverity = 0.0f;
1293     boundarySeverity = 0.0f;
1294     bool leftActive = false;
1295     bool rightActive = false;
1296     for (int i = 0; i < 8; i++) {
1297         if (sensor[i] != HIGH) continue;
1298         float weight = BOUNDARY_STEER_WEIGHTS[i];
1299         float severity = fabsf(weight);
1300         weightedSum += weight;
1301         activeBoundarySensors++;
1302         if (severity > boundarySeverity) boundarySeverity = severity;
1303         if (i <= 3) {
1304             leftActive = true;
1305             if (severity > leftBoundarySeverity) leftBoundarySeverity = severity;
1306         }
1307         else {
1308             rightActive = true;
1309             if (severity > rightBoundarySeverity) rightBoundarySeverity = severity;
1310         }
1311     }
1312     boundaryDetected = activeBoundarySensors > 0;
1313     centreBoundaryDetected = ( sensor[3] == HIGH || sensor[4] == HIGH );
1314     bothSidesBoundaryDetected = leftActive && rightActive;
1315     if (!boundaryDetected) {
1316         boundarySteerError = 0.0f;
1317         return;
```

Integrated autonomous controller

ORIGINAL LINES 1,318-1,356

```
1318     }
1319     boundarySteerError = weightedSum / (float)activeBoundarySensors;
1320     // Keep a useful directional memory. Do not replace it
1321     // with an almost perfectly balanced / ambiguous value.
1322     if ( fabsf(boundarySteerError) > 0.25f ) {
1323         lastBoundarySteerError = boundarySteerError;
1324     }
1325 }
1326 // =====
1327 // TOF READ
1328 // =====
1329 bool readToFDistance( uint8_t address, uint16_t &distanceMM ) {
1330     Wire.beginTransaction(address);
1331     Wire.write( (uint8_t)( SIM_RANGE_REGISTER >> 8 ) );
1332     Wire.write( (uint8_t)( SIM_RANGE_REGISTER & 0xFF ) );
1333     // NORMAL STOP TRANSACTION
1334     uint8_t result = Wire.endTransmission();
1335     if (result != 0) return false;
1336     uint8_t received = Wire.requestFrom( address, (uint8_t)2 );
1337     if ( received != 2 || Wire.available() < 2 ) {
1338         return false;
1339     }
1340     uint8_t highByte = Wire.read();
1341     uint8_t lowByte = Wire.read();
1342     distanceMM = ((uint16_t)highByte << 8) | lowByte;
1343     // =====
1344     // VL53L1X SENSOR-FAULT DETECTION
1345     // The Wokwi fault simulator returns 0xFFFF (65535)
1346     // when Sensor Fault = 1. A zero reading is also
1347     // treated as invalid.
1348     // =====
1349     if ( distanceMM == 0 || distanceMM >= 5000 ) {
1350         return false;
1351     }
1352     return true;
1353 }
1354 // =====
1355 // TOF + TTC UPDATE
1356 // =====
```

Integrated autonomous controller

ORIGINAL LINES 1,357-1,395

```
1357 void updateToF() {
1358     unsigned long now = millis();
1359     if ( now - previousToFTime < TOF_INTERVAL_MS ) {
1360         return;
1361     }
1362     float dt = (now - previousToFTime) / 1000.0f;
1363     previousToFTime = now;
1364     uint16_t newLeft = 0;
1365     uint16_t newRight = 0;
1366     bool leftOK = false;
1367     bool rightOK = false;
1368     if (USE_WOKWI_INTERNAL_TOF_SIM) {
1369         // Internal ToF mode is retained only as a fallback.
1370         // The current Wokwi race build keeps this FALSE so the
1371         // custom VL53L1X distance sliders are used.
1372         newLeft = 1500;
1373         newRight = 1500;
1374         leftOK = true;
1375         rightOK = true;
1376     }
1377     else {
1378         // Interactive Wokwi mode:
1379         // read the actual custom VL53L1X chips.
1380         // Moving either distance slider changes these values.
1381         leftOK = readToFDistance( LEFT_TOF_ADDR, newLeft );
1382         rightOK = readToFDistance( RIGHT_TOF_ADDR, newRight );
1383     }
1384     if (!leftOK || !rightOK) {
1385         tofValid = false;
1386         previousToFValid = false;
1387         return;
1388     }
1389     tofValid = true;
1390     leftDistance = newLeft;
1391     rightDistance = newRight;
1392     if ( previousToFValid && dt > 0.001f ) {
1393         leftClosingSpeed = ( (float)previousLeftDistance - (float)leftDistance ) / dt;
1394         rightClosingSpeed = ( (float)previousRightDistance - (float)rightDistance ) / dt;
1395         if ( leftClosingSpeed > MIN_CLOSING_SPEED_MM_S ) {
```

Integrated autonomous controller

ORIGINAL LINES 1,396-1,434

```
1396     leftTTC = (float)leftDistance / leftClosingSpeed;
1397 }
1398 else {
1399     leftTTC = -1.0f;
1400 }
1401 if ( rightClosingSpeed > MIN_CLOSING_SPEED_MM_S ) {
1402     rightTTC = (float)rightDistance / rightClosingSpeed;
1403 }
1404 else {
1405     rightTTC = -1.0f;
1406 }
1407 }
1408 else {
1409     leftClosingSpeed = 0.0f;
1410     rightClosingSpeed = 0.0f;
1411     leftTTC = -1.0f;
1412     rightTTC = -1.0f;
1413 }
1414 previousLeftDistance = leftDistance;
1415 previousRightDistance = rightDistance;
1416 previousToFValid = true;
1417 }
1418 // =====
1419 // INA219 REGISTER READ
1420 // Uses WRITE -> STOP -> READ -> STOP.
1421 // This matches the stable transaction style already used
1422 // by the shared Wokwi I2C simulation.
1423 // =====
1424 bool readINA219Register( uint8_t reg, uint16_t &value ) {
1425     Wire.beginTransmission( INA219_ADDR );
1426     Wire.write(reg);
1427     if ( Wire.endTransmission() != 0 ) {
1428         return false;
1429     }
1430     uint8_t received = Wire.requestFrom( INA219_ADDR, (uint8_t)2 );
1431     if ( received != 2 || Wire.available() < 2 ) {
1432         return false;
1433     }
1434     uint8_t highByte = Wire.read();
```

Integrated autonomous controller

ORIGINAL LINES 1,435-1,473

```
1435     uint8_t lowByte = Wire.read();
1436     value = ((uint16_t)highByte << 8) | lowByte;
1437     return true;
1438 }
1439 // =====
1440 // INA219 UPDATE
1441 // =====
1442 void updateINA219( bool forceRead = false ) {
1443     unsigned long now = millis();
1444     if ( !forceRead && now - previousINATime < INA_INTERVAL_MS ) {
1445         return;
1446     }
1447     previousINATime = now;
1448     if (USE_WOKWI_INTERNAL_INA_SIM) {
1449         inaValid = WOKWI_INA_ONLINE;
1450         if (!inaValid) {
1451             motorStallDetected = false;
1452             stallConditionStart = 0;
1453             return;
1454         }
1455         busVoltageV = WOKWI_BUS_VOLTAGE_V;
1456         motorCurrentMA = WOKWI_MOTOR_CURRENT_MA;
1457         return;
1458     }
1459     uint16_t rawBusVoltage = 0;
1460     uint16_t rawCurrent = 0;
1461     bool busOK = readINA219Register( INA219_REG_BUS_VOLTAGE, rawBusVoltage );
1462     bool currentOK = readINA219Register( INA219_REG_CURRENT, rawCurrent );
1463     if (!busOK || !currentOK) {
1464         inaValid = false;
1465         motorStallDetected = false;
1466         stallConditionStart = 0;
1467         return;
1468     }
1469     inaValid = true;
1470     // INA219-style bus voltage register:
1471     // voltage bits are [15:3], 4 mV per bit.
1472     uint16_t busVoltageMV = (rawBusVoltage >> 3) * 4U;
1473     busVoltageV = (float)busVoltageMV / 1000.0f;
```

Integrated autonomous controller

ORIGINAL LINES 1,474-1,512

```
1474 // Wokwi behavioural-model convention:
1475 // 1 raw current count = 1 mA.
1476 motorCurrentMA = (float)( (int16_t)rawCurrent );
1477 }
1478 // =====
1479 // MOTOR STALL DETECTION
1480 // =====
1481 // A stall is only declared when ALL are true:
1482 // 1. INA219 current is high.
1483 // 2. The controller is commanding meaningful motion.
1484 // 3. Actual wheel speed is very low.
1485 // 4. The condition persists for STALL_CONFIRM_MS.
1486 // The time confirmation prevents startup or brief
1487 // transients from immediately causing a false stall.
1488 // =====
1489 void updateMotorStallDetection() {
1490 // =====
1491 // HARD STALL LATCH
1492 // =====
1493 // Once a real stall is confirmed, current falling back
1494 // to normal MUST NOT automatically restart the robot.
1495 // The robot stays stopped until the operator deliberately
1496 // cycles the MAIN switch:
1497 //   MAIN ON -> stall detected -> latched stop
1498 //   MAIN OFF -> latch is cleared while motors are disabled
1499 //   MAIN ON  -> autonomous control may start again
1500 // This is safer for a fully autonomous racing robot because
1501 // it prevents repeated automatic attempts against a jammed
1502 // wheel or trapped drivetrain.
1503 // =====
1504 if (motorStallLatched) {
1505     motorStallDetected = true;
1506     return;
1507 }
1508 if (!inaValid) {
1509     motorStallDetected = false;
1510     stallCandidateMask = 0;
1511     stallLatchedSideMask = 0;
1512     stallConditionStart = 0;
```


Integrated autonomous controller

ORIGINAL LINES 1,513-1,551

```
1513     return;
1514 }
1515 // Do not diagnose a mechanical stall from corrupt
1516 // encoder samples.
1517 if (!leftRPMValid || !rightRPMValid) {
1518     motorStallDetected = false;
1519     stallCandidateMask = 0;
1520     stallConditionStart = 0;
1521     return;
1522 }
1523 // =====
1524 // PER-WHEEL STALL CHECK
1525 // =====
1526 // Previous logic averaged the two wheel RPM values.
1527 // That misses a one-wheel stall:
1528 //   left = 0 RPM
1529 //   right = 38 RPM
1530 //   average = 19 RPM
1531 // 19 RPM is above the 5 RPM stall threshold, so the
1532 // old controller incorrectly concluded "NO STALL".
1533 // We now evaluate each wheel independently.
1534 // =====
1535 bool highCurrent = motorCurrentMA >= STALL_CURRENT_MA;
1536 bool leftCommanded = fabsf( leftRampedTargetRPM ) > STALL_COMMAND_RPM_THRESHOLD;
1537 bool rightCommanded = fabsf( rightRampedTargetRPM ) > STALL_COMMAND_RPM_THRESHOLD;
1538 bool leftTooSlow = fabsf( leftRPM ) < STALL_RPM_THRESHOLD;
1539 bool rightTooSlow = fabsf( rightRPM ) < STALL_RPM_THRESHOLD;
1540 bool leftStallCondition = highCurrent && leftCommanded && leftTooSlow;
1541 bool rightStallCondition = highCurrent && rightCommanded && rightTooSlow;
1542 stallCandidateMask = 0;
1543 if (leftStallCondition) {
1544     stallCandidateMask |= 0x01;
1545 }
1546 if (rightStallCondition) {
1547     stallCandidateMask |= 0x02;
1548 }
1549 bool stallCondition = stallCandidateMask != 0;
1550 unsigned long now = millis();
1551 if (!stallCondition) {
```

Integrated autonomous controller

ORIGINAL LINES 1,552-1,590

```
1552     motorStallDetected = false;
1553     stallConditionStart = 0;
1554     return;
1555 }
1556 if (stallConditionStart == 0) {
1557     stallConditionStart = now;
1558     motorStallDetected = false;
1559     return;
1560 }
1561 if ( now - stallConditionStart >= STALL_CONFIRM_MS ) {
1562     motorStallLatched = true;
1563     motorStallDetected = true;
1564     stallLatchedSideMask = stallCandidateMask;
1565 }
1566 else {
1567     motorStallDetected = false;
1568 }
1569 }
1570 void resetMotorStallLatchWhileSystemOff() {
1571     // This function must only be called while MAIN SWITCH = OFF.
1572     // The motor driver is already disabled in SYSTEM OFF, so it
1573     // is safe to clear the software latch here. The robot cannot
1574     // move again until the operator deliberately switches MAIN ON.
1575     motorStallLatched = false;
1576     motorStallDetected = false;
1577     stallCandidateMask = 0;
1578     stallLatchedSideMask = 0;
1579     stallConditionStart = 0;
1580 }
1581 const char *stallSideName() {
1582     uint8_t mask = motorStallLatched ? stallLatchedSideMask : stallCandidateMask;
1583     if (mask == 0x03) {
1584         return "BOTH";
1585     }
1586     if (mask == 0x01) {
1587         return "LEFT";
1588     }
1589     if (mask == 0x02) {
1590         return "RIGHT";
```

Integrated autonomous controller

ORIGINAL LINES 1,591-1,629

```
1591     }
1592     return "NONE";
1593 }
1594 // =====
1595 // CURRENT STATUS TEXT
1596 // =====
1597 const char *currentStatusName() {
1598     if (!inaValid) return "SENSOR ERROR";
1599     if (motorStallDetected) return "STALL";
1600     if ( motorCurrentMA >= HIGH_CURRENT_MA ) {
1601         return "HIGH";
1602     }
1603     return "NORMAL";
1604 }
1605 // =====
1606 // MPU WRITE
1607 // =====
1608 bool writeMPURegister( uint8_t reg, uint8_t value ) {
1609     Wire.beginTransaction( MPU_ADDR );
1610     Wire.write(reg);
1611     Wire.write(value);
1612     return Wire.endTransmission() == 0;
1613 }
1614 // =====
1615 // MPU READ BYTE
1616 // IMPORTANT:
1617 // NO REPEATED START.
1618 // We deliberately use:
1619 // WRITE -> STOP
1620 // READ  -> STOP
1621 // because this is stable with the current Wokwi
1622 // shared I2C simulation.
1623 // =====
1624 bool readMPUByte( uint8_t reg, uint8_t &value ) {
1625     Wire.beginTransaction( MPU_ADDR );
1626     Wire.write(reg);
1627     if ( Wire.endTransmission() != 0 ) {
1628         return false;
1629     }
}
```

Integrated autonomous controller

ORIGINAL LINES 1,630-1,668

```
1630     uint8_t received = Wire.requestFrom( MPU_ADDR, (uint8_t)1 );
1631     if ( received != 1 || Wire.available() < 1 ) {
1632         return false;
1633     }
1634     value = Wire.read();
1635     return true;
1636 }
1637 // =====
1638 // MPU INITIALISATION
1639 // =====
1640 bool initialiseMPU() {
1641     if (USE_WOKWI_INTERNAL_IMU_SIM) {
1642         gyroZ = WOKWI_GYRO_Z_DPS;
1643         return WOKWI_IMU_ONLINE;
1644     }
1645     if ( !writeMPURegister( MPU_PWR_MGMT_1, 0x00 ) ) {
1646         return false;
1647     }
1648     if ( !writeMPURegister( MPU_GYRO_CONFIG, 0x00 ) ) {
1649         return false;
1650     }
1651     uint8_t who = 0;
1652     if ( !readMPUByte( MPU_WHO_AM_I, who ) ) {
1653         return false;
1654     }
1655     return who == 0x68;
1656 }
1657 // =====
1658 // READ GYRO Z
1659 // ALSO USES NORMAL STOP TRANSACTIONS.
1660 // =====
1661 bool readGyroZ() {
1662     if (USE_WOKWI_INTERNAL_IMU_SIM) {
1663         gyroZ = WOKWI_GYRO_Z_DPS;
1664         pitchDeg = wokwiSimulatedPitchDeg;
1665         return WOKWI_IMU_ONLINE;
1666     }
1667     // Physical MPU6050:
1668     // read accel XYZ + temp + gyro XYZ in one 14-byte burst.
```

Integrated autonomous controller

ORIGINAL LINES 1,669-1,707

```
1669 Wire.beginTransmission( MPU_ADDR );
1670 Wire.write( MPU_ACCEL_XOUT_H );
1671 if ( Wire.endTransmission() != 0 ) {
1672     return false;
1673 }
1674 uint8_t received = Wire.requestFrom( MPU_ADDR, (uint8_t)14 );
1675 if ( received != 14 || Wire.available() < 14 ) {
1676     return false;
1677 }
1678 int16_t rawAx = ((int16_t)Wire.read() << 8) | Wire.read();
1679 int16_t rawAy = ((int16_t)Wire.read() << 8) | Wire.read();
1680 int16_t rawAz = ((int16_t)Wire.read() << 8) | Wire.read();
1681 // Temperature (unused).
1682 Wire.read();
1683 Wire.read();
1684 // Gyro X and Y (unused here).
1685 Wire.read();
1686 Wire.read();
1687 Wire.read();
1688 Wire.read();
1689 int16_t rawGz = ((int16_t)Wire.read() << 8) | Wire.read();
1690 gyroZ = (float)rawGz / GYRO_SENSITIVITY;
1691 // Assumed physical orientation:
1692 // X points forward, Z points upward.
1693 // Change axis/sign after real mounting calibration if needed.
1694 float ax = (float)rawAx;
1695 float ay = (float)rawAy;
1696 float az = (float)rawAz;
1697 float measuredPitch = atan2f( -ax, sqrtf( ay * ay + az * az ) ) * 180.0f / PI;
1698 // Low-pass pitch for vibration/noise.
1699 pitchDeg = 0.80f * pitchDeg + 0.20f * measuredPitch;
1700 return true;
1701 }
1702 // =====
1703 // TTC HELPERS
1704 // =====
1705 float getMinimumTTC() {
1706     bool leftValid = leftTTC > 0.0f;
1707     bool rightValid = rightTTC > 0.0f;
```

Integrated autonomous controller

ORIGINAL LINES 1,708-1,746

```
1708     if (leftValid && rightValid) {
1709         return leftTTC < rightTTC ? leftTTC :
1710             rightTTC;
1711     }
1712     if (leftValid) return leftTTC;
1713     if (rightValid) return rightTTC;
1714     return 999.0f;
1715 }
1716 // =====
1717 // AVOIDANCE DIRECTION
1718 // -1 = LEFT
1719 // +1 = RIGHT
1720 // 0 = UNDETERMINED
1721 // =====
1722 int chooseAvoidanceDirection() {
1723     int direction = 0;
1724     // First choose the safer side using TTC when available.
1725     if ( leftTTC > 0.0f && rightTTC > 0.0f ) {
1726         if ( fabsf( leftTTC - rightTTC ) > 0.15f ) {
1727             if (leftTTC < rightTTC) direction = +1; // obstacle closes sooner on left -> go right
1728             else direction = -1; // obstacle closes sooner on right -> go left
1729         }
1730     }
1731     else if (leftTTC > 0.0f) {
1732         direction = +1;
1733     }
1734     else if (rightTTC > 0.0f) {
1735         direction = -1;
1736     }
1737     // If TTC did not decide, use the measured distances.
1738     if (direction == 0) {
1739         int difference = (int)leftDistance - (int)rightDistance;
1740         if ( abs(difference) > SIDE_TOLERANCE_MM ) {
1741             if (leftDistance < rightDistance) direction = +1;
1742             else direction = -1;
1743         }
1744     }
1745     // =====
1746     // ROAD-BOUNDARY VETO
```

Integrated autonomous controller

ORIGINAL LINES 1,747-1,785

```
1747 // =====
1748 // Do not avoid an obstacle by steering directly into
1749 // a nearby road boundary.
1750 // direction -1 = LEFT
1751 // direction +1 = RIGHT
1752 // Severity >= 2 means the boundary has reached beyond
1753 // the far-outside sensor and we treat that side as
1754 // unsafe for an obstacle-avoidance turn.
1755 // =====
1756 if ( direction < 0 && leftBoundarySeverity >= 2.0f ) {
1757     return 0;
1758 }
1759 if ( direction > 0 && rightBoundarySeverity >= 2.0f ) {
1760     return 0;
1761 }
1762 return direction;
1763 }
1764 // =====
1765 // SKID DETECTION
1766 // =====
1767 void updateSkidDetection() {
1768     if (!leftRPMValid || !rightRPMValid) {
1769         possibleSkid = false;
1770         return;
1771     }
1772     float rpmDifference = fabsf( leftRPM - rightRPM );
1773     float averageRPM = ( fabsf(leftRPM) + fabsf(rightRPM) ) / 2.0f;
1774     possibleSkid = ( averageRPM > 10.0f && rpmDifference < 3.0f && fabsf(gyroZ) > 25.0f );
1775 }
1776 // =====
1777 // AUTONOMOUS BRIDGE PHASE ESTIMATION
1778 // =====
1779 const char *bridgePhaseName() {
1780     switch (bridgePhase) {
1781         case BRIDGE_NONE:     return "NONE";
1782         case BRIDGE_APPROACH: return "APPROACH";
1783         case BRIDGE_CLIMB:    return "CLIMB";
1784         case BRIDGE_CREST:    return "CREST";
1785         case BRIDGE_DESCENT:  return "DESCENT";
```

Integrated autonomous controller

ORIGINAL LINES 1,786-1,824

```
1786     case BRIDGE_EXIT:      return "EXIT";
1787 }
1788 return "INVALID";
1789 }
1790 void updateBridgePhase() {
1791     if (pitchDeg > maximumBridgeClimbPitchDeg) {
1792         maximumBridgeClimbPitchDeg = pitchDeg;
1793     }
1794     switch (bridgePhase) {
1795     case BRIDGE_NONE:
1796         maximumBridgeClimbPitchDeg = 0.0f;
1797         bridgeExitStartTime = 0;
1798         if ( visionIsTrusted() && visionBridgeConfidencePct >= BRIDGE_CAMERA_ENTER_CONFIDENCE_PCT ) {
1799             bridgePhase = BRIDGE_APPROACH;
1800             return;
1801         }
1802         // IMU fallback if camera prediction is missed.
1803         if ( imuValid && pitchDeg >= BRIDGE_CLIMB_ENTER_PITCH_DEG ) {
1804             bridgePhase = BRIDGE_CLIMB;
1805             maximumBridgeClimbPitchDeg = pitchDeg;
1806             return;
1807         }
1808         if ( imuValid && pitchDeg <= BRIDGE_DESCENT_ENTER_PITCH_DEG ) {
1809             bridgePhase = BRIDGE_DESCENT;
1810             return;
1811         }
1812         break;
1813     case BRIDGE_APPROACH:
1814         if ( imuValid && pitchDeg >= BRIDGE_CLIMB_ENTER_PITCH_DEG ) {
1815             bridgePhase = BRIDGE_CLIMB;
1816             maximumBridgeClimbPitchDeg = pitchDeg;
1817             return;
1818         }
1819         // Cancel false camera bridge detection.
1820         if ( visionBridgeConfidencePct < BRIDGE_CAMERA_CLEAR_CONFIDENCE_PCT && fabsf(pitchDeg) < BRIDGE_FLAT_PITCH_DEG ) {
1821             bridgePhase = BRIDGE_NONE;
1822             return;
1823         }
1824         break;
```


Integrated autonomous controller

ORIGINAL LINES 1,825-1,863

```
1825     case BRIDGE_CLIMB:
1826         if ( pitchDeg <= BRIDGE_DESCENT_ENTER_PITCH_DEG ) {
1827             bridgePhase = BRIDGE_DESCENT;
1828             return;
1829         }
1830         if ( maximumBridgeClimbPitchDeg >= BRIDGE_CLIMB_ENTER_PITCH_DEG && fabsf(pitchDeg) <= BRIDGE_CREST_PITCH_DEG ) {
1831             bridgePhase = BRIDGE_CREST;
1832             return;
1833         }
1834         break;
1835     case BRIDGE_CREST:
1836         if ( pitchDeg <= BRIDGE_DESCENT_ENTER_PITCH_DEG ) {
1837             bridgePhase = BRIDGE_DESCENT;
1838             return;
1839         }
1840         if ( pitchDeg >= BRIDGE_CLIMB_ENTER_PITCH_DEG ) {
1841             bridgePhase = BRIDGE_CLIMB;
1842             return;
1843         }
1844         break;
1845     case BRIDGE_DESCENT:
1846         if ( fabsf(pitchDeg) <= BRIDGE_FLAT_PITCH_DEG ) {
1847             bridgePhase = BRIDGE_EXIT;
1848             bridgeExitStartTime = millis();
1849             return;
1850         }
1851         break;
1852     case BRIDGE_EXIT:
1853         if ( pitchDeg <= BRIDGE_DESCENT_ENTER_PITCH_DEG ) {
1854             bridgePhase = BRIDGE_DESCENT;
1855             bridgeExitStartTime = 0;
1856             return;
1857         }
1858         if ( fabsf(pitchDeg) <= BRIDGE_FLAT_PITCH_DEG && visionBridgeConfidencePct < BRIDGE_CAMERA_CLEAR_CONFIDENCE_PCT ) {
1859             if ( millis() - bridgeExitStartTime >= BRIDGE_EXIT_CONFIRM_MS ) {
1860                 bridgePhase = BRIDGE_NONE;
1861                 maximumBridgeClimbPitchDeg = 0.0f;
1862                 bridgeExitStartTime = 0;
1863                 return;

```

Integrated autonomous controller

ORIGINAL LINES 1,864-1,900

```
1864     }
1865   }
1866   else {
1867     bridgeExitStartTime = millis();
1868   }
1869   break;
1870 }
1871 }
1872 // =====
1873 // BRIDGE-SAFE CAMERA STEERING
1874 // =====
1875 void applyBridgeGuidance( float baseRPM, float maximumSteerRPM ) {
1876   if (!visionIsTrusted()) {
1877     applyRoadKeeping( baseRPM );
1878     return;
1879   }
1880   float safeBridgeBaseRPM = min( baseRPM, getVisionCornerSpeedCapRPM() );
1881   float normalizedSteer = constrain( (float)visionSteerPct / 100.0f, -1.0f, 1.0f );
1882   float correctionRPM = maximumSteerRPM * normalizedSteer;
1883   leftTargetRPM = constrain( safeBridgeBaseRPM + correctionRPM, MIN_TARGET_RPM, min( MAX_TARGET_RPM, safeBridgeBaseRPM + maximumSteerRPM ) );
1884   rightTargetRPM = constrain( safeBridgeBaseRPM - correctionRPM, MIN_TARGET_RPM, min( MAX_TARGET_RPM, safeBridgeBaseRPM + maximumSteerRPM ) );
1885 }
1886 // =====
1887 // STATE DECISION
1888 // =====
1889 void determineNavigationState() {
1890   // =====
1891   // 1. ABSOLUTE SAFETY: E-STOP
1892   // =====
1893   if (emergencyStopActive) {
1894     currentState = STATE_EMERGENCY_STOP;
1895     return;
1896   }
1897   // =====
1898   // 2. MAIN SWITCH OFF
1899   // =====
1900   if (!mainSwitchOn) {
```

Integrated autonomous controller

ORIGINAL LINES 1,901-1,939

```
1901     currentState = STATE_SYSTEM_OFF;
1902     return;
1903 }
1904 // =====
1905 // 3. MOTOR / ELECTRICAL PROTECTION
1906 // =====
1907 if (motorStallDetected) {
1908     currentState = STATE_MOTOR_STALL;
1909     return;
1910 }
1911 // =====
1912 // 3. COLLISION SAFETY: ToF + TTC
1913 // =====
1914 // Other race cars and track walls are dynamic hazards.
1915 // TTC makes the decision depend on closing rate as well
1916 // as raw distance.
1917 // =====
1918 if (tofValid) {
1919     uint16_t nearestDistance = leftDistance < rightDistance ? leftDistance :
1920         rightDistance;
1921     float minimumTTC = getMinimumTTC();
1922     int avoidDirection = chooseAvoidanceDirection();
1923     // Critical collision risk
1924     if ( nearestDistance <= CRITICAL_RANGE_MM || minimumTTC <= TTC_CRITICAL_S ) {
1925         if (avoidDirection > 0) currentState = STATE_CRITICAL_AVOID_RIGHT;
1926         else if (avoidDirection < 0) currentState = STATE_CRITICAL_AVOID_LEFT;
1927         else currentState = STATE_CRITICAL_STOP;
1928         return;
1929     }
1930     // Active avoidance
1931     if ( nearestDistance <= AVOID_RANGE_MM || minimumTTC <= TTC_AVOID_S ) {
1932         if (avoidDirection > 0) currentState = STATE_AVOID_RIGHT;
1933         else if (avoidDirection < 0) currentState = STATE_AVOID_LEFT;
1934         else currentState = STATE_NO_SAFE_PATH;
1935         return;
1936     }
1937     // Early caution
1938     if ( nearestDistance <= CAUTION_RANGE_MM || minimumTTC <= TTC_CAUTION_S ) {
1939         currentState = STATE_CAUTION;
```

Integrated autonomous controller

ORIGINAL LINES 1,940-1,978

```
1940     return;
1941 }
1942 }
1943 // =====
1944 // 4. SKID / STABILITY PROTECTION
1945 // =====
1946 if (possibleSkid) {
1947     currentState = STATE_SKID_CAUTION;
1948     return;
1949 }
1950 // =====
1951 // 5. IR ROAD-BOUNDARY OVERRIDE
1952 // =====
1953 // Camera predicts what is coming.
1954 // IR protects what is happening under the vehicle NOW.
1955 // =====
1956 if (boundaryDetected) {
1957     if ( centreBoundaryDetected && fabsf(boundarySteerError) <= 0.25f ) {
1958         currentState = STATE_BOUNDARY_ESCAPE;
1959         return;
1960     }
1961     if (boundarySteerError > 0.25f) {
1962         currentState = STATE_BOUNDARY_CORRECT_RIGHT;
1963         return;
1964     }
1965     if (boundarySteerError < -0.25f) {
1966         currentState = STATE_BOUNDARY_CORRECT_LEFT;
1967         return;
1968     }
1969     currentState = STATE_BOUNDARY_CAUTION;
1970     return;
1971 }
1972 // =====
1973 // 6. ToF FAULT -> DEGRADED SPEED
1974 // =====
1975 // Camera + IR can still navigate, but full racing speed
1976 // is not allowed without the car/wall collision layer.
1977 // =====
1978 if (!tofValid) {
```

Integrated autonomous controller

ORIGINAL LINES 1,979-2,017

```
1979     currentState = STATE_SENSOR_FAULT;
1980     return;
1981 }
1982 // =====
1983 // 8. AUTONOMOUS BRIDGE HANDLING
1984 // =====
1985 switch (bridgePhase) {
1986     case BRIDGE_APPROACH:
1987         currentState = STATE_BRIDGE_APPROACH;
1988         return;
1989     case BRIDGE_CLIMB:
1990         currentState = STATE_BRIDGE_CLIMB;
1991         return;
1992     case BRIDGE_CREST:
1993         currentState = STATE_BRIDGE_CREST;
1994         return;
1995     case BRIDGE_DESCENT:
1996         currentState = STATE_BRIDGE_DESCENT;
1997         return;
1998     case BRIDGE_EXIT:
1999         currentState = STATE_BRIDGE_EXIT;
2000         return;
2001     case BRIDGE_NONE:
2002         break;
2003 }
2004 // =====
2005 // 9. CAMERA PREDICTIVE LOOKAHEAD
2006 // =====
2007 if (visionIsTrusted()) {
2008     currentState = STATE_VISION_DRIVE;
2009     return;
2010 }
2011 // Camera missing / stale / low confidence.
2012 currentState = STATE_VISION_FALLBACK;
2013 }
2014 // =====
2015 // CHOOSE BOUNDARY ESCAPE DIRECTION
2016 // =====
2017 // Return:
```

Integrated autonomous controller

ORIGINAL LINES 2,018-2,056

```
2018 // -1 = turn LEFT
2019 // +1 = turn RIGHT
2020 // This is mainly for an ambiguous centre-boundary hit
2021 // such as 00011000. We first use directional memory,
2022 // then use ToF free space, then use a deterministic
2023 // right-turn fallback for the Wokwi test.
2024 // =====
2025 int chooseBoundaryEscapeDirection() {
2026     if (lastBoundarySteerError > 0.25f) return +1;
2027     if (lastBoundarySteerError < -0.25f) return -1;
2028     if (tofValid) {
2029         if ( leftDistance > rightDistance + SIDE_TOLERANCE_MM ) {
2030             return -1;
2031         }
2032         if ( rightDistance > leftDistance + SIDE_TOLERANCE_MM ) {
2033             return +1;
2034         }
2035     }
2036     return +1;
2037 }
2038 // =====
2039 // ROAD KEEPING / BOUNDARY AVOIDANCE
2040 // =====
2041 void applyRoadKeeping( float requestedBaseRPM ) {
2042     // Clear road:
2043     // all IR sensors are over the drivable road surface.
2044     if (!boundaryDetected) {
2045         leftTargetRPM = requestedBaseRPM;
2046         rightTargetRPM = requestedBaseRPM;
2047         return;
2048     }
2049     // Reduce speed as the boundary moves closer to the
2050     // centre of the sensor array.
2051     float baseRPM = requestedBaseRPM;
2052     if (boundarySeverity >= 3.5f) {
2053         baseRPM = min( baseRPM, BOUNDARY_CENTRE_RPM );
2054     }
2055     else if (boundarySeverity >= 2.5f) {
2056         baseRPM = min( baseRPM, BOUNDARY_INNER_RPM );
```

Integrated autonomous controller

ORIGINAL LINES 2,057-2,095

```
2057     }
2058     else if (bothSidesBoundaryDetected) {
2059         baseRPM = min( baseRPM, BOUNDARY_CAUTION_RPM );
2060     }
2061     float correction = K_BOUNDARY_STEER * boundarySteerError;
2062     // Positive correction means the boundary is on the
2063     // LEFT, therefore the left wheel speeds up and the
2064     // right wheel slows down to turn the robot RIGHT.
2065     leftTargetRPM = baseRPM + correction;
2066     rightTargetRPM = baseRPM - correction;
2067     leftTargetRPM = constrain( leftTargetRPM, MIN_TARGET_RPM, MAX_TARGET_RPM );
2068     rightTargetRPM = constrain( rightTargetRPM, MIN_TARGET_RPM, MAX_TARGET_RPM );
2069 }
2070 // =====
2071 // BOUNDARY ESCAPE
2072 // =====
2073 void applyBoundaryEscape() {
2074     int direction = chooseBoundaryEscapeDirection();
2075     if (direction > 0) {
2076         // Turn RIGHT strongly.
2077         leftTargetRPM = AVOID_FAST_RPM;
2078         rightTargetRPM = CRITICAL_AVOID_SLOW_RPM;
2079     }
2080     else {
2081         // Turn LEFT strongly.
2082         leftTargetRPM = CRITICAL_AVOID_SLOW_RPM;
2083         rightTargetRPM = AVOID_FAST_RPM;
2084     }
2085 }
2086 // =====
2087 // APPLY NAVIGATION TARGETS
2088 // =====
2089 void applyNavigationTargets() {
2090     switch (currentState) {
2091         case STATE_EMERGENCY_STOP:
2092         case STATE_SYSTEM_OFF:
2093             leftTargetRPM = 0.0f;
2094             rightTargetRPM = 0.0f;
2095             break;
```

Integrated autonomous controller

ORIGINAL LINES 2,096-2,134

```
2096     case STATE_VISION_DRIVE:
2097         applyVisionGuidance();
2098         break;
2099     case STATE_VISION_FALLBACK:
2100         applyRoadKeeping( VISION_FALLBACK_RPM );
2101         break;
2102     case STATE_ROAD_DRIVE:
2103         applyRoadKeeping( getVisionBaseRPM() );
2104         break;
2105     case STATE_BOUNDARY_CORRECT_LEFT:
2106         applyRoadKeeping( getVisionBaseRPM() );
2107         break;
2108     case STATE_BOUNDARY_CORRECT_RIGHT:
2109         applyRoadKeeping( getVisionBaseRPM() );
2110         break;
2111     case STATE_BOUNDARY_CAUTION:
2112         applyRoadKeeping( min( getVisionBaseRPM(), BOUNDARY_CAUTION_RPM ) );
2113         break;
2114     case STATE_BOUNDARY_ESCAPE:
2115         applyBoundaryEscape();
2116         break;
2117     case STATE_CAUTION:
2118         // Preserve road/camera guidance while reducing speed.
2119         if (boundaryDetected) {
2120             applyRoadKeeping( CAUTION_BASE_RPM );
2121         }
2122         else if (visionIsTrusted()) {
2123             applyVisionGuidanceAtBase( CAUTION_BASE_RPM );
2124         }
2125         else {
2126             leftTargetRPM = CAUTION_BASE_RPM;
2127             rightTargetRPM = CAUTION_BASE_RPM;
2128         }
2129         break;
2130     case STATE_AVOID_LEFT:
2131         leftTargetRPM = AVOID_SLOW_RPM;
2132         rightTargetRPM = AVOID_FAST_RPM;
2133         break;
2134     case STATE_AVOID_RIGHT:
```


Integrated autonomous controller

ORIGINAL LINES 2,135-2,173

```
2135     leftTargetRPM = AVOID_FAST_RPM;
2136     rightTargetRPM = AVOID_SLOW_RPM;
2137     break;
2138 case STATE_CRITICAL_AVOID_LEFT:
2139     leftTargetRPM = CRITICAL_AVOID_SLOW_RPM;
2140     rightTargetRPM = CRITICAL_AVOID_FAST_RPM;
2141     break;
2142 case STATE_CRITICAL_AVOID_RIGHT:
2143     leftTargetRPM = CRITICAL_AVOID_FAST_RPM;
2144     rightTargetRPM = CRITICAL_AVOID_SLOW_RPM;
2145     break;
2146 case STATE_CRITICAL_STOP:
2147     leftTargetRPM = 0.0f;
2148     rightTargetRPM = 0.0f;
2149     break;
2150 case STATE_NO_SAFE_PATH:
2151     leftTargetRPM = 0.0f;
2152     rightTargetRPM = 0.0f;
2153     break;
2154 case STATE_MOTOR_STALL:
2155     leftTargetRPM = 0.0f;
2156     rightTargetRPM = 0.0f;
2157     break;
2158 case STATE_SKID_CAUTION:
2159     if (boundaryDetected) {
2160         applyRoadKeeping( SKID_BASE_RPM );
2161     }
2162     else if (visionIsTrusted()) {
2163         applyVisionGuidanceAtBase( SKID_BASE_RPM );
2164     }
2165     else {
2166         leftTargetRPM = SKID_BASE_RPM;
2167         rightTargetRPM = SKID_BASE_RPM;
2168     }
2169     break;
2170 case STATE_BRIDGE_APPROACH:
2171     applyBridgeGuidance( BRIDGE_APPROACH_RPM, BRIDGE_APPROACH_STEER_RPM );
2172     break;
2173 case STATE_BRIDGE_CLIMB:
```

Integrated autonomous controller

ORIGINAL LINES 2,174-2,212

```
2174     applyBridgeGuidance( BRIDGE_CLIMB_RPM, BRIDGE_CLIMB_STEER_RPM );
2175     break;
2176 case STATE_BRIDGE_CREST:
2177     applyBridgeGuidance( BRIDGE_CREST_RPM, BRIDGE_CREST_STEER_RPM );
2178     break;
2179 case STATE_BRIDGE_DESCENT:
2180     applyBridgeGuidance( BRIDGE_DESCENT_RPM, BRIDGE_DESCENT_STEER_RPM );
2181     break;
2182 case STATE_BRIDGE_EXIT:
2183     applyBridgeGuidance( BRIDGE_EXIT_RPM, BRIDGE_EXIT_STEER_RPM );
2184     break;
2185 case STATE_SENSOR_FAULT:
2186     if (boundaryDetected) {
2187         applyRoadKeeping( SENSOR_FAULT_RPM );
2188     }
2189     else if (visionIsTrusted()) {
2190         applyVisionGuidanceAtBase( SENSOR_FAULT_RPM );
2191     }
2192     else {
2193         leftTargetRPM = SENSOR_FAULT_RPM;
2194         rightTargetRPM = SENSOR_FAULT_RPM;
2195     }
2196     break;
2197 default:
2198     leftTargetRPM = 0.0f;
2199     rightTargetRPM = 0.0f;
2200     break;
2201 }
2202 }
2203 // =====
2204 // STATE CHANGE HANDLING
2205 // =====
2206 void handleStateChange() {
2207     if ( currentState == previousState ) {
2208         return;
2209     }
2210     leftIntegral = 0.0f;
2211     rightIntegral = 0.0f;
2212     leftPreviousError = 0.0f;
```

Integrated autonomous controller

ORIGINAL LINES 2,213-2,251

```
2213     rightPreviousError = 0.0f;
2214     previousState = currentState;
2215 }
2216 // =====
2217 // IMMEDIATE HARD-STOP OUTPUT
2218 // =====
2219 bool isHardStopState() {
2220     return currentState == STATE_EMERGENCY_STOP || currentState == STATE_SYSTEM_OFF || currentState == STATE_CRITICAL_STOP ||
2221         currentState == STATE_NO_SAFE_PATH || currentState == STATE_MOTOR_STALL;
2222 }
2223 void applyHardStopOutputs() {
2224     leftTargetRPM = 0.0f;
2225     rightTargetRPM = 0.0f;
2226     leftRampedTargetRPM = 0.0f;
2227     rightRampedTargetRPM = 0.0f;
2228     leftPWM = 0;
2229     rightPWM = 0;
2230     leftIntegral = 0.0f;
2231     rightIntegral = 0.0f;
2232     digitalWrite(PWMA, LOW);
2233     digitalWrite(PWMB, LOW);
2234 }
2235 // =====
2236 // FAST RACE / SAFETY CONTROL
2237 // =====
2238 // This is the fast steering loop.
2239 // It reads ONLY the IR boundary array and updates the
2240 // navigation target quickly. It does not wait for the
2241 // slower RPM/PID cycle.
2242 // This is important for a fast robot because waiting
2243 // 250 ms before noticing a road edge would be far too
2244 // slow at racing speed.
2245 // =====
2246 void updateFastRoadControl() {
2247     unsigned long now = millis();
2248     if ( now - previousRoadControlTime < ROAD_CONTROL_INTERVAL_MS ) {
2249         return;
2250     }
2251     previousRoadControlTime = now;
```

Integrated autonomous controller

ORIGINAL LINES 2,252-2,290

```
2252     updateWokwiBridgeScenario();
2253     updateVisionSerial();
2254     updateBridgePhase();
2255     // Internally rate-limited to 20 Hz.
2256     updateToF();
2257     readBoundarySensors();
2258     analyseRoadBoundaries();
2259     determineNavigationState();
2260     enforceValidNavigationState();
2261     handleStateChange();
2262     applyNavigationTargets();
2263     if (isHardStopState()) {
2264         applyHardStopOutputs();
2265     }
2266 }
2267 // =====
2268 // SPEED RAMP UPDATE
2269 // =====
2270 float rampTowardTarget( float currentValue, float requestedValue, float dt ) {
2271     if (dt <= 0.0f) return currentValue;
2272     float difference = requestedValue - currentValue;
2273     if (fabsf(difference) < 0.001f) return requestedValue;
2274     // Speed increases use the gentle acceleration rate.
2275     // Speed reductions use the faster deceleration rate.
2276     float rate = ( fabsf(requestedValue) > fabsf(currentValue) ) ? ACCEL_RAMP_RPM_PER_SEC :
2277         DECEL_RAMP_RPM_PER_SEC;
2278     float maximumStep = rate * dt;
2279     if (difference > maximumStep) return currentValue + maximumStep;
2280     if (difference < -maximumStep) return currentValue - maximumStep;
2281     return requestedValue;
2282 }
2283 void updateSpeedRamp( float dt ) {
2284     // Safety stops NEVER use a ramp.
2285     if (isHardStopState()) {
2286         leftRampedTargetRPM = 0.0f;
2287         rightRampedTargetRPM = 0.0f;
2288         return;
2289     }
2290     leftRampedTargetRPM = rampTowardTarget( leftRampedTargetRPM, leftTargetRPM, dt );
```

Integrated autonomous controller

ORIGINAL LINES 2,291-2,329

```
2291     rightRampedTargetRPM = rampTowardTarget( rightRampedTargetRPM, rightTargetRPM, dt );
2292 }
2293 // =====
2294 // RPM SAMPLE VALIDATION
2295 // =====
2296 bool isValidRPMSample(float rpm) {
2297     return isfinite(rpm) && fabsf(rpm) <= MAX_VALID_RPM;
2298 }
2299 // =====
2300 // PID UPDATE
2301 // =====
2302 void updatePID() {
2303     unsigned long now = millis();
2304     if ( now - previousPIDTime < PID_INTERVAL_MS ) {
2305         return;
2306     }
2307     float dt = ( now - previousPIDTime ) / 1000.0f;
2308     previousPIDTime = now;
2309     // RPM
2310     long leftDelta = leftEncoderCount - previousLeftPIDCount;
2311     long rightDelta = rightEncoderCount - previousRightPIDCount;
2312     previousLeftPIDCount = leftEncoderCount;
2313     previousRightPIDCount = rightEncoderCount;
2314     float newLeftRPM = ( (float)leftDelta / CPR ) * ( 60.0f / dt );
2315     float newRightRPM = ( (float)rightDelta / CPR ) * ( 60.0f / dt );
2316     // Reject NaN, infinity, or an impossible Wokwi RPM spike.
2317     // If a bad sample occurs, keep the previous valid RPM so
2318     // one corrupt encoder sample cannot destabilise the PID.
2319     leftRPMValid = isValidRPMSample( newLeftRPM );
2320     rightRPMValid = isValidRPMSample( newRightRPM );
2321     if ( leftRPMValid ) {
2322         leftRawRPM = newLeftRPM;
2323         if ( !leftRPMFilterInitialised ) {
2324             leftRPM = newLeftRPM;
2325             leftRPMFilterInitialised = true;
2326         }
2327     } else {
2328         leftRPM = RPM_FILTER_ALPHA * newLeftRPM + (1.0f - RPM_FILTER_ALPHA) * leftRPM;
2329     }
```

Integrated autonomous controller

ORIGINAL LINES 2,330-2,368

```
2330     }
2331     if (rightRPMValid) {
2332         rightRawRPM = newRightRPM;
2333         if (!rightRPMFilterInitialised) {
2334             rightRPM = newRightRPM;
2335             rightRPMFilterInitialised = true;
2336         }
2337         else {
2338             rightRPM = RPM_FILTER_ALPHA * newRightRPM + (1.0f - RPM_FILTER_ALPHA) * rightRPM;
2339         }
2340     }
2341     // SLOWER HEALTH / STABILITY PERCEPTION
2342     // Camera, IR and ToF are handled by the fast race loop.
2343     // This PID cycle updates current sensing and IMU data.
2344     updateINA219();
2345     imuValid = readGyroZ();
2346     if (!imuValid) {
2347         gyroZ = 0.0f;
2348         pitchDeg = 0.0f;
2349     }
2350     updateBridgePhase();
2351     updateSkidDetection();
2352     updateMotorStallDetection();
2353     // SAFETY / STABILITY DECISION
2354     // Re-run the state selector after the slower health
2355     // sensors update. Motor stall and skid states can then
2356     // override normal road driving.
2357     determineNavigationState();
2358     enforceValidNavigationState();
2359     handleStateChange();
2360     applyNavigationTargets();
2361     // Apply the physics-aware acceleration/deceleration ramp
2362     // before the PID calculates motor error.
2363     updateSpeedRamp( dt );
2364     // HARD STOP
2365     if (isHardStopState()) {
2366         leftPWM = 0;
2367         rightPWM = 0;
2368         leftIntegral = 0.0f;
```

Integrated autonomous controller

ORIGINAL LINES 2,369-2,407

```
2369     rightIntegral = 0.0f;
2370     digitalWrite(PWMA, LOW);
2371     digitalWrite(PWMB, LOW);
2372     return;
2373 }
2374 // =====
2375 // POSITIONAL PID + FEED-FORWARD + ANTI-WINDUP
2376 // =====
2377 // Previous controller:
2378 //   PWM = old PWM + correction
2379 // That could accumulate output and overshoot.
2380 // New controller:
2381 //   PWM = feed-forward + current PID correction
2382 // The output is recalculated fresh every PID cycle.
2383 // =====
2384 float leftError = leftRampedTargetRPM - leftRPM;
2385 float rightError = rightRampedTargetRPM - rightRPM;
2386 float leftDerivative = ( leftError - leftPreviousError ) / dt;
2387 float rightDerivative = ( rightError - rightPreviousError ) / dt;
2388 leftPreviousError = leftError;
2389 rightPreviousError = rightError;
2390 float leftCandidateIntegral = constrain( leftIntegral + leftError * dt, -PID_INTEGRAL_LIMIT, PID_INTEGRAL_LIMIT );
2391 float rightCandidateIntegral = constrain( rightIntegral + rightError * dt, -PID_INTEGRAL_LIMIT, PID_INTEGRAL_LIMIT );
2392 float leftFeedForward = leftRampedTargetRPM * PWM_FEEDFORWARD_PER_RPM;
2393 float rightFeedForward = rightRampedTargetRPM * PWM_FEEDFORWARD_PER_RPM;
2394 float leftCandidateOutput = leftFeedForward + Kp * leftError + Ki * leftCandidateIntegral + Kd * leftDerivative;
2395 float rightCandidateOutput = rightFeedForward + Kp * rightError + Ki * rightCandidateIntegral + Kd * rightDerivative;
2396 bool allowLeftIntegral = ( ( leftCandidateOutput >= 0.0f && leftCandidateOutput <= 255.0f ) || ( leftCandidateOutput > 255.0f &&
2397     leftError < 0.0f ) || ( leftCandidateOutput < 0.0f && leftError > 0.0f ) );
2398 bool allowRightIntegral = ( ( rightCandidateOutput >= 0.0f && rightCandidateOutput <= 255.0f ) || ( rightCandidateOutput > 255.0f &&
2399     rightError < 0.0f ) || ( rightCandidateOutput < 0.0f && rightError > 0.0f ) );
2400 if (allowLeftIntegral) {
2401     leftIntegral = leftCandidateIntegral;
2402 }
2403 if (allowRightIntegral) {
2404     rightIntegral = rightCandidateIntegral;
2405 }
2406 if ( leftRPM > leftRampedTargetRPM + OVERSPEED_MARGIN_RPM && leftIntegral > 0.0f ) {
2407     leftIntegral *= OVERSPEED_INTEGRAL_BLEED;
```

Integrated autonomous controller

ORIGINAL LINES 2,408-2,446

```
2408     }
2409     if ( rightRPM > rightRampedTargetRPM + OVERSPEED_MARGIN_RPM && rightIntegral > 0.0f ) {
2410         rightIntegral *= OVERSPEED_INTEGRAL_BLEED;
2411     }
2412     float leftPWMCommand = leftFeedForward + Kp * leftError + Ki * leftIntegral + Kd * leftDerivative;
2413     float rightPWMCommand = rightFeedForward + Kp * rightError + Ki * rightIntegral + Kd * rightDerivative;
2414     leftPWM = constrain( (int)roundf( leftPWMCommand ), 0, 255 );
2415     rightPWM = constrain( (int)roundf( rightPWMCommand ), 0, 255 );
2416 }
2417 // =====
2418 // NAVIGATION STATE VALIDATION / FAIL-SAFE
2419 // =====
2420 // currentState should only contain one of the enum values
2421 // assigned by determineNavigationState().
2422 // If an invalid value ever appears, fail SAFE:
2423 //   invalid state -> CRITICAL STOP -> PWM 0 / 0
2424 // =====
2425 bool navigationStateIsValid() {
2426     switch (currentState) {
2427         case STATE_EMERGENCY_STOP:
2428         case STATE_VISION_DRIVE:
2429         case STATE_VISION_FALLBACK:
2430         case STATE_ROAD_DRIVE:
2431         case STATE_BOUNDARY_CORRECT_LEFT:
2432         case STATE_BOUNDARY_CORRECT_RIGHT:
2433         case STATE_BOUNDARY_CAUTION:
2434         case STATE_BOUNDARY_ESCAPE:
2435         case STATE_CAUTION:
2436         case STATE_AVOID_LEFT:
2437         case STATE_AVOID_RIGHT:
2438         case STATE_CRITICAL_AVOID_LEFT:
2439         case STATE_CRITICAL_AVOID_RIGHT:
2440         case STATE_CRITICAL_STOP:
2441         case STATE_NO_SAFE_PATH:
2442         case STATE_MOTOR_STALL:
2443         case STATE_SKID_CAUTION:
2444         case STATE_SENSOR_FAULT:
2445         case STATE_SYSTEM_OFF:
2446         case STATE_BRIDGE_APPROACH:
```


Integrated autonomous controller

ORIGINAL LINES 2,447-2,485

```
2447     case STATE_BRIDGE_CLIMB:
2448     case STATE_BRIDGE_CREST:
2449     case STATE_BRIDGE_DESCENT:
2450     case STATE_BRIDGE_EXIT:
2451         return true;
2452     }
2453     return false;
2454 }
2455 void enforceValidNavigationState() {
2456     if (navigationStateIsValid()) {
2457         return;
2458     }
2459     invalidStateRecoveryCount++;
2460     currentState = STATE_CRITICAL_STOP;
2461     leftTargetRPM = 0.0f;
2462     rightTargetRPM = 0.0f;
2463     leftRampedTargetRPM = 0.0f;
2464     rightRampedTargetRPM = 0.0f;
2465     leftPWM = 0;
2466     rightPWM = 0;
2467     leftIntegral = 0.0f;
2468     rightIntegral = 0.0f;
2469     digitalWrite(PWMA, LOW);
2470     digitalWrite(PWMB, LOW);
2471 }
2472 // =====
2473 // STATE NAME
2474 // =====
2475 const char *stateName() {
2476     switch (currentState) {
2477     case STATE_EMERGENCY_STOP:
2478         return "EMERGENCY STOP";
2479     case STATE_VISION_DRIVE:
2480         return "VISION PREDICTIVE DRIVE";
2481     case STATE_VISION_FALLBACK:
2482         return "VISION FALLBACK - IR ONLY";
2483     case STATE_ROAD_DRIVE:
2484         return "ROAD DRIVE";
2485     case STATE_BOUNDARY_CORRECT_LEFT:
```

Integrated autonomous controller

ORIGINAL LINES 2,486-2,524

```
2486     return "BOUNDARY -> CORRECT LEFT";
2487 case STATE_BOUNDARY_CORRECT_RIGHT:
2488     return "BOUNDARY -> CORRECT RIGHT";
2489 case STATE_BOUNDARY_CAUTION:
2490     return "BOUNDARY CAUTION";
2491 case STATE_BOUNDARY_ESCAPE:
2492     return "BOUNDARY ESCAPE";
2493 case STATE_CAUTION:
2494     return "TOF CAUTION";
2495 case STATE_AVOID_LEFT:
2496     return "AVOID LEFT";
2497 case STATE_AVOID_RIGHT:
2498     return "AVOID RIGHT";
2499 case STATE_CRITICAL_AVOID_LEFT:
2500     return "CRITICAL AVOID LEFT";
2501 case STATE_CRITICAL_AVOID_RIGHT:
2502     return "CRITICAL AVOID RIGHT";
2503 case STATE_CRITICAL_STOP:
2504     return "CRITICAL STOP";
2505 case STATE_NO_SAFE_PATH:
2506     return "NO SAFE PATH - STOP";
2507 case STATE_MOTOR_STALL:
2508     return "MOTOR STALL";
2509 case STATE_SKID_CAUTION:
2510     return "POSSIBLE SKID";
2511 case STATE_SENSOR_FAULT:
2512     return "TOF SENSOR FAULT - DEGRADED";
2513 case STATE_SYSTEM_OFF:
2514     return "SYSTEM OFF";
2515 case STATE_BRIDGE_APPROACH:
2516     return "BRIDGE APPROACH";
2517 case STATE_BRIDGE_CLIMB:
2518     return "BRIDGE CLIMB";
2519 case STATE_BRIDGE_CREST:
2520     return "BRIDGE CREST";
2521 case STATE_BRIDGE_DESCENT:
2522     return "BRIDGE DESCENT";
2523 case STATE_BRIDGE_EXIT:
2524     return "BRIDGE EXIT";
```

Integrated autonomous controller

ORIGINAL LINES 2,525-2,562

```
2525     }
2526     return "INVALID STATE - FAILSAFE";
2527 }
2528 // =====
2529 // PRINT TTC
2530 // =====
2531 void printTTCValue(float value) {
2532     if (value < 0.0f) {
2533         DebugSerial.print("N/A");
2534     }
2535     else {
2536         DebugSerial.print(value, 2);
2537         DebugSerial.print(" s");
2538     }
2539 }
2540 // =====
2541 // DIAGNOSTICS
2542 // =====
2543 void printDiagnostics() {
2544     unsigned long now = millis();
2545     if ( now - previousPrintTime < PRINT_INTERVAL_MS ) {
2546         return;
2547     }
2548     previousPrintTime = now;
2549     DebugSerial.println();
2550     DebugSerial.println( "===== " );
2551     DebugSerial.print( "RUN MODE          : " );
2552     DebugSerial.println( RUNNING_IN_WOKWI ? "WOKWI SCALED RPM" : ( PHYSICAL_INITIAL_TEST_PROFILE ? "PHYSICAL INITIAL TEST" : "PHYSICAL R
ACE" ) );
2553     DebugSerial.print( "NORMAL RPM LIMIT : " );
2554     DebugSerial.println( NORMAL_BASE_RPM, 1 );
2555     DebugSerial.print( "STATE          : " );
2556     DebugSerial.println( stateName() );
2557     DebugSerial.print( "STATE CODE       : " );
2558     DebugSerial.println( (int)currentState );
2559     DebugSerial.print( "STATE FAILSAFE CNT: " );
2560     DebugSerial.println( invalidStateRecoveryCount );
2561     DebugSerial.print( "E-STOP STATUS    : " );
2562     DebugSerial.println( emergencyStopActive ? "PRESSED" : "RELEASED" );
```

Integrated autonomous controller

ORIGINAL LINES 2,563-2,601

```
2563   DebugSerial.print( "MAIN SWITCH      : " );
2564   DebugSerial.println( mainSwitchOn ? "ON" : "OFF" );
2565   DebugSerial.print( "START INTERLOCK  : " );
2566   if (raceStartInterlockCleared) {
2567       DebugSerial.println( "RACE ENABLED" );
2568   }
2569   else if (!mainSwitchOn) {
2570       DebugSerial.println( "READY - SWITCH ON TO START" );
2571   }
2572   else {
2573       DebugSerial.println( "LOCKED - SWITCH OFF FIRST" );
2574   }
2575   DebugSerial.print( "IR BOUNDARY      : " );
2576   for (int i = 0; i < 8; i++) {
2577       DebugSerial.print(sensor[i]);
2578   }
2579   DebugSerial.println();
2580   DebugSerial.print( "ROAD STATUS      : " );
2581   if (!boundaryDetected) {
2582       DebugSerial.println( "CLEAR" );
2583   }
2584   else if ( centreBoundaryDetected && fabsf(boundarySteerError) <= 0.25f ) {
2585       DebugSerial.println( "CENTRE BOUNDARY / ESCAPE" );
2586   }
2587   else if ( boundarySteerError > 0.25f ) {
2588       DebugSerial.println( "LEFT BOUNDARY - STEER RIGHT" );
2589   }
2590   else if ( boundarySteerError < -0.25f ) {
2591       DebugSerial.println( "RIGHT BOUNDARY - STEER LEFT" );
2592   }
2593   else {
2594       DebugSerial.println( "BOTH SIDES / BALANCED" );
2595   }
2596   DebugSerial.print( "VISION SOURCE      : " );
2597   DebugSerial.println( USE_WOKWI_INTERNAL_VISION_SIM ? "WOKWI INTERNAL SIM" : "RASPBERRY PI UART" );
2598   DebugSerial.print( "VISION STATUS      : " );
2599   if (visionIsTrusted()) {
2600       DebugSerial.println( "ONLINE / TRUSTED" );
2601   }
```

Integrated autonomous controller

ORIGINAL LINES 2,602-2,640

```
2602     else if (visionIsOnline()) {
2603         DebugSerial.println( "ONLINE / LOW CONFIDENCE" );
2604     }
2605     else {
2606         DebugSerial.println( "OFFLINE / STALE" );
2607     }
2608     DebugSerial.print( "VISION STEER      : " );
2609     DebugSerial.print( visionSteerPct );
2610     DebugSerial.println( " %" );
2611     DebugSerial.print( "VISION SPEED      : " );
2612     DebugSerial.print( visionSpeedPct );
2613     DebugSerial.println( " %" );
2614     DebugSerial.print( "VISION CONFIDENCE : " );
2615     DebugSerial.print( visionConfidencePct );
2616     DebugSerial.println( " %" );
2617     DebugSerial.print( "BRIDGE CAM CONF : " );
2618     DebugSerial.print( visionBridgeConfidencePct );
2619     DebugSerial.println( " %" );
2620     DebugSerial.print( "CORNER TEST      : " );
2621     DebugSerial.println( WOKWI_AUTO_CORNER_SPEED_TEST ? "ACTIVE / INDEPENDENT" : "DISABLED" );
2622     DebugSerial.print( "CORNER SEVERITY : " );
2623     DebugSerial.print( visionCornerSeverityPct );
2624     DebugSerial.println( " %" );
2625     DebugSerial.print( "CORNER CLASS      : " );
2626     DebugSerial.println( cornerClassName() );
2627     DebugSerial.print( "CORNER RPM CAP : " );
2628     DebugSerial.println( getVisionCornerSpeedCapRPM(), 2 );
2629     DebugSerial.print( "VISION BASE RPM : " );
2630     DebugSerial.println( getVisionBaseRPM(), 2 );
2631     DebugSerial.print( "PI RX BYTES : " );
2632     DebugSerial.println( piRxByteCount );
2633     DebugSerial.print( "PI GOOD PACKETS : " );
2634     DebugSerial.println( piGoodPacketCount );
2635     DebugSerial.print( "PI BAD PACKETS : " );
2636     DebugSerial.println( piBadPacketCount );
2637     DebugSerial.print( "PI LAST RAW : " );
2638     DebugSerial.println( piLastRawPacket );
2639     DebugSerial.print( "BOUNDARY STEER : " );
2640     DebugSerial.println( boundarySteerError, 2 );
```

Integrated autonomous controller

ORIGINAL LINES 2,641-2,679

```
2641   DebugSerial.print( "LAST BOUNDARY      : " );
2642   DebugSerial.println( lastBoundarySteerError, 2 );
2643   DebugSerial.print( "BOUNDARY SEVERITY : " );
2644   DebugSerial.println( boundarySeverity, 1 );
2645   DebugSerial.println();
2646   DebugSerial.print( "TOF SOURCE          : " );
2647   if (USE_WOKWI_INTERNAL_TOF_SIM) {
2648       DebugSerial.println( "WOKWI INTERNAL" );
2649   }
2650   else if (USE_WOKWI_TOF_SLIDER_CHIPS) {
2651       DebugSerial.println( "VL53L1X SLIDERS" );
2652   }
2653   else {
2654       DebugSerial.println( "PHYSICAL I2C" );
2655   }
2656   DebugSerial.print( "IMU SOURCE          : " );
2657   DebugSerial.println( USE_WOKWI_INTERNAL_IMU_SIM ? "WOKWI INTERNAL" : "PHYSICAL I2C" );
2658   DebugSerial.print( "INA SOURCE          : " );
2659   if (USE_WOKWI_INTERNAL_INA_SIM) {
2660       DebugSerial.println( "WOKWI INTERNAL" );
2661   }
2662   else if (USE_WOKWI_INA219_SLIDER_CHIP) {
2663       DebugSerial.println( "INA219 SLIDERS" );
2664   }
2665   else {
2666       DebugSerial.println( "PHYSICAL I2C" );
2667   }
2668   DebugSerial.print( "LEFT TOF          : " );
2669   if (tofValid) {
2670       DebugSerial.print(leftDistance);
2671       DebugSerial.println(" mm");
2672   }
2673   else {
2674       DebugSerial.println("INVALID");
2675   }
2676   DebugSerial.print( "RIGHT TOF         : " );
2677   if (tofValid) {
2678       DebugSerial.print(rightDistance);
2679       DebugSerial.println(" mm");
```

Integrated autonomous controller

ORIGINAL LINES 2,680-2,718

```
2680     }
2681     else {
2682         DebugSerial.println("INVALID");
2683     }
2684     DebugSerial.print( "LEFT CLOSING      : " );
2685     DebugSerial.print(leftClosingSpeed, 1);
2686     DebugSerial.println(" mm/s");
2687     DebugSerial.print( "RIGHT CLOSING     : " );
2688     DebugSerial.print(rightClosingSpeed, 1);
2689     DebugSerial.println(" mm/s");
2690     DebugSerial.print( "LEFT TTC          : " );
2691     printTTCValue(leftTTC);
2692     DebugSerial.println();
2693     DebugSerial.print( "RIGHT TTC         : " );
2694     printTTCValue(rightTTC);
2695     DebugSerial.println();
2696     DebugSerial.println();
2697     DebugSerial.print( "TRACK MODE       : " );
2698     DebugSerial.println( "CAMERA + IR + TOF RACING" );
2699     DebugSerial.print( "OBSTACLE MODE      : " );
2700     DebugSerial.println( OBSTACLE_AVOIDANCE_ENABLED ? "ENABLED" : "ENABLED - CARS / WALLS" );
2701     DebugSerial.print( "ROAD LOOP         : " );
2702     DebugSerial.print( 1000UL / ROAD_CONTROL_INTERVAL_MS );
2703     DebugSerial.println( " Hz" );
2704     DebugSerial.println();
2705     DebugSerial.print( "INA219 STATUS      : " );
2706     DebugSerial.println( inaValid ? "ONLINE" : "READ ERROR" );
2707     DebugSerial.print( "BUS VOLTAGE        : " );
2708     if (inaValid) {
2709         DebugSerial.print( busVoltageV, 2 );
2710         DebugSerial.println( " V" );
2711     }
2712     else {
2713         DebugSerial.println( "N/A" );
2714     }
2715     DebugSerial.print( "MOTOR CURRENT      : " );
2716     if (inaValid) {
2717         DebugSerial.print( motorCurrentMA, 0 );
2718         DebugSerial.println( " mA" );
```

Integrated autonomous controller

ORIGINAL LINES 2,719-2,757

```
2719     }
2720     else {
2721         DebugSerial.println( "N/A" );
2722     }
2723     DebugSerial.print( "CURRENT STATUS      : " );
2724     DebugSerial.println( currentStatusName() );
2725     DebugSerial.print( "STALL FLAG          : " );
2726     DebugSerial.println( motorStallDetected ? "YES" : "NO" );
2727     DebugSerial.print( "STALL LATCH          : " );
2728     DebugSerial.println( motorStallLatched ? "LOCKED" : "CLEAR" );
2729     DebugSerial.print( "STALL SIDE          : " );
2730     DebugSerial.println( stallSideName() );
2731     DebugSerial.print( "STALL RESET          : " );
2732     DebugSerial.println( motorStallLatched ? "MAIN SWITCH OFF -> ON REQUIRED" : "READY" );
2733     DebugSerial.println();
2734     DebugSerial.print( "MPU STATUS          : " );
2735     DebugSerial.println( imuValid ? "ONLINE" : "READ ERROR" );
2736     DebugSerial.print( "GYRO Z / YAW          : " );
2737     DebugSerial.print( gyroZ, 2 );
2738     DebugSerial.println( " deg/s" );
2739     DebugSerial.print( "PITCH          : " );
2740     DebugSerial.print( pitchDeg, 2 );
2741     DebugSerial.println( " deg" );
2742     DebugSerial.print( "BRIDGE PHASE          : " );
2743     DebugSerial.println( bridgePhaseName() );
2744     DebugSerial.print( "SKID FLAG          : " );
2745     DebugSerial.println( possibleSkid ? "YES" : "NO" );
2746     DebugSerial.println();
2747     DebugSerial.print( "LEFT REQUEST RPM      : " );
2748     DebugSerial.println( leftTargetRPM, 2 );
2749     DebugSerial.print( "RIGHT REQUEST RPM     : " );
2750     DebugSerial.println( rightTargetRPM, 2 );
2751     DebugSerial.print( "LEFT PID SETPOINT     : " );
2752     DebugSerial.println( leftRampedTargetRPM, 2 );
2753     DebugSerial.print( "RIGHT PID SETPOINT    : " );
2754     DebugSerial.println( rightRampedTargetRPM, 2 );
2755     DebugSerial.print( "LEFT ACTUAL RPM       : " );
2756     if (leftRPMValid) {
2757         DebugSerial.println( leftRPM, 2 );
```


Integrated autonomous controller

ORIGINAL LINES 2,758-2,796

```
2758     }
2759     else {
2760         DebugSerial.println( "INVALID" );
2761     }
2762     DebugSerial.print( "RIGHT ACTUAL RPM : " );
2763     if (rightRPMValid) {
2764         DebugSerial.println( rightRPM, 2 );
2765     }
2766     else {
2767         DebugSerial.println( "INVALID" );
2768     }
2769     DebugSerial.print( "LEFT RAW RPM      : " );
2770     DebugSerial.println( leftRawRPM, 2 );
2771     DebugSerial.print( "RIGHT RAW RPM     : " );
2772     DebugSerial.println( rightRawRPM, 2 );
2773     DebugSerial.print( "RPM SAMPLE STATUS : " );
2774     DebugSerial.println( (leftRPMValid && rightRPMValid) ? "OK" : "CHECK ENCODERS" );
2775     DebugSerial.println( "PID MODE          : POSITIONAL + FF + ANTI-WINDUP" );
2776     DebugSerial.print( "LEFT PWM           : " );
2777     DebugSerial.println(leftPWM);
2778     DebugSerial.print( "RIGHT PWM          : " );
2779     DebugSerial.println(rightPWM);
2780     // Ensure the complete diagnostics frame leaves the UART buffer
2781     DebugSerial.flush();
2782 }
2783 // =====
2784 // SETUP
2785 // =====
2786 void setup() {
2787     DebugSerial.begin(115200);
2788     PiSerial.begin(115200);
2789     // Allow the first ToF update during setup.
2790     previousToFTime = millis() - TOF_INTERVAL_MS;
2791     delay(500);
2792     // IMPORTANT:
2793     // Print immediately so we know setup has started.
2794     DebugSerial.println();
2795     DebugSerial.println( "===== " );
2796     DebugSerial.println( " ROBO RUMBLE - FULL SENSOR RACE CONTROLLER" );
```

Integrated autonomous controller

ORIGINAL LINES 2,797-2,835

```
2797     DebugSerial.println( "===== " );
2798     DebugSerial.println();
2799     DebugSerial.println( "[BOOT] Setup started" );
2800     DebugSerial.println( "[BOOT] Pi vision UART: PB11 RX / PB10 TX @ 115200" );
2801     DebugSerial.print( "[BOOT] Vision source: " );
2802     DebugSerial.println( USE_WOKWI_INTERNAL_VISION_SIM ? "WOKWI INTERNAL SIM" : "RASPBERRY PI UART" );
2803     DebugSerial.println( "[BOOT] Pi RX diagnostics enabled" );
2804     DebugSerial.println( "[BOOT] FINAL INTEGRATED MODE: scripted tests DISABLED" );
2805     DebugSerial.println( "[BOOT] STARTUP INTERLOCK: MAIN must be OFF, then switched ON" );
2806     DebugSerial.println( "[BOOT] MAIN SWITCH: 60 ms DEBOUNCE + ONE-SHOT RACE START" );
2807     DebugSerial.println( "[BOOT] Sensor sources:" );
2808     DebugSerial.print( "          ToF : " );
2809     if (USE_WOKWI_INTERNAL_TOF_SIM) {
2810         DebugSerial.println( "WOKWI INTERNAL" );
2811     }
2812     else if (USE_WOKWI_TOF_SLIDER_CHIPS) {
2813         DebugSerial.println( "VL53L1X SLIDERS" );
2814     }
2815     else {
2816         DebugSerial.println( "PHYSICAL I2C" );
2817     }
2818     DebugSerial.print( "          IMU : " );
2819     DebugSerial.println( USE_WOKWI_INTERNAL_IMU_SIM ? "WOKWI INTERNAL" : "PHYSICAL I2C" );
2820     DebugSerial.print( "          INA : " );
2821     DebugSerial.println( USE_WOKWI_INTERNAL_INA_SIM ? "WOKWI INTERNAL" : "PHYSICAL I2C" );
2822     // EMERGENCY STOP
2823     pinMode( ESTOP_PIN, INPUT_PULLUP );
2824     readEmergencyStop();
2825     // MOTOR PINS
2826     pinMode(PWMA, OUTPUT);
2827     pinMode(AIN1, OUTPUT);
2828     pinMode(AIN2, OUTPUT);
2829     pinMode(PWMB, OUTPUT);
2830     pinMode(BIN1, OUTPUT);
2831     pinMode(BIN2, OUTPUT);
2832     pinMode(STBY, OUTPUT);
2833     // SAFE BOOT OUTPUTS:
2834     // Before any sensor initialisation or race logic, both PWM
2835     // outputs and the motor driver are forced OFF.
```

Integrated autonomous controller

ORIGINAL LINES 2,836-2,874

```
2836     digitalWrite(PWMA, LOW);
2837     digitalWrite(PWMB, LOW);
2838     digitalWrite(STBY, LOW);
2839     // ENCODERS
2840     pinMode(LEFT_ENC_A, INPUT);
2841     pinMode(LEFT_ENC_B, INPUT);
2842     pinMode(RIGHT_ENC_A, INPUT);
2843     pinMode(RIGHT_ENC_B, INPUT);
2844     // IR
2845     for (int i = 0; i < 8; i++) {
2846         pinMode( IR_PINS[i], INPUT );
2847     }
2848     // TOF XSHUT
2849     // PB3/PB4 are used as GPIO for the two ToF shutdown pins.
2850     // On physical STM32F103 hardware these pins overlap with
2851     // the default JTAG interface. If supported by the core,
2852     // disable JTAG while retaining SWD.
2853     // For this Wokwi model the XSHUT lines stay HIGH because
2854     // the two simulated sensors already have unique addresses.
2855     // The race-safe custom ToF chip deliberately ignores live
2856     // XSHUT disconnects to avoid a Wokwi custom-I2C bus lockup.
2857     #ifdef __HAL_AFIO_REMAP_SWJ_NOJTAG
2858         __HAL_RCC_AFIO_CLK_ENABLE();
2859         __HAL_AFIO_REMAP_SWJ_NOJTAG();
2860     #endif
2861     // MAIN ON/OFF SWITCH
2862     // PA15 is usable as GPIO after JTAG is disabled.
2863     pinMode( MAIN_SWITCH_PIN, INPUT_PULLUP );
2864     // Initialise the debounced MAIN-switch state.
2865     // This NEVER starts the race at boot, even if the switch is ON.
2866     initialiseMainSwitch();
2867     pinMode( LEFT_XSHUT, OUTPUT );
2868     pinMode( RIGHT_XSHUT, OUTPUT );
2869     // =====
2870     // WOKWI-SAFE XSHUT INITIALISATION
2871     // =====
2872     // IMPORTANT:
2873     // Our two custom Wokwi VL53L1X chips already have fixed
2874     // simulation addresses (0x30 and 0x31).
```

Integrated autonomous controller

ORIGINAL LINES 2,875-2,913

```
2875 // Toggling a custom I2C device off and back on with
2876 // XSHUT can make Wokwi's shared custom-I2C bus hang.
2877 // Therefore, in Wokwi we simply keep both sensors
2878 // enabled from startup.
2879 // The PHYSICAL robot is different: real VL53L1X
2880 // sensors normally start at the same default address,
2881 // so physical firmware will later sequence XSHUT and
2882 // assign unique I2C addresses during startup.
2883 // =====
2884 digitalWrite( LEFT_XSHUT, HIGH );
2885 digitalWrite( RIGHT_XSHUT, HIGH );
2886 DebugSerial.println( "[BOOT] GPIO configured" );
2887 // I2C
2888 // The bus remains configured even when the integrated
2889 // Wokwi internal-sensor mode is enabled. This preserves
2890 // the physical pin map in one codebase.
2891 Wire.setSDA(PB7);
2892 Wire.setSCL(PB6);
2893 Wire.begin();
2894 Wire.setClock( 100000 );
2895 DebugSerial.println( "[BOOT] I2C started" );
2896 delay(100);
2897 // TOF
2898 // Do not perform a VL53L1X transaction inside setup().
2899 // The custom Wokwi I2C pair is serviced by the normal
2900 // race loop after all devices have finished booting.
2901 // This also keeps startup deterministic: the Serial
2902 // Monitor reaches READY before the first ToF sample.
2903 DebugSerial.println( "[BOOT] VL53L1X pair: DEFERRED TO RACE LOOP" );
2904 // MPU
2905 DebugSerial.println( "[BOOT] Initialising MPU6050..." );
2906 imuValid = initialiseMPU();
2907 DebugSerial.print( "[BOOT] MPU6050: " );
2908 DebugSerial.println( imuValid ? "ONLINE" : "FAILED" );
2909 // INA219
2910 DebugSerial.println( "[BOOT] Checking INA219..." );
2911 updateINA219( true );
2912 DebugSerial.print( "[BOOT] INA219: " );
2913 DebugSerial.println( inaValid ? "ONLINE" : "FAILED" );
```

Integrated autonomous controller

ORIGINAL LINES 2,914-2,951

```
2914 // ENCODER INITIAL STATES
2915 previousLeftEncoderState = ( digitalRead(LEFT_ENC_A) << 1 ) | digitalRead(LEFT_ENC_B);
2916 previousRightEncoderState = ( digitalRead(RIGHT_ENC_A) << 1 ) | digitalRead(RIGHT_ENC_B);
2917 DebugSerial.println( "[BOOT] Encoders configured" );
2918 // MOTORS
2919 motorsForward();
2920 if (emergencyStopActive) {
2921     applyEmergencyStopImmediately();
2922     DebugSerial.println( "[BOOT] E-STOP: PRESSED - MOTOR DRIVER DISABLED" );
2923 }
2924 else {
2925     DebugSerial.println( "[BOOT] E-STOP: RELEASED" );
2926 }
2927 DebugSerial.print( "[BOOT] MAIN SWITCH: " );
2928 DebugSerial.println( mainSwitchOn ? "ON" : "OFF" );
2929 pwmPeriodStart = micros();
2930 previousPIDTime = millis();
2931 previousRoadControlTime = millis();
2932 previousToFTime = millis() - TOF_INTERVAL_MS;
2933 previousINATime = millis() - INA_INTERVAL_MS;
2934 DebugSerial.println( "[BOOT] Motor driver enabled" );
2935 DebugSerial.print( "[BOOT] Speed ramp: ACCEL " );
2936 DebugSerial.print( ACCEL_RAMP_RPM_PER_SEC, 1 );
2937 DebugSerial.print( " RPM/s | DECEL " );
2938 DebugSerial.print( DECEL_RAMP_RPM_PER_SEC, 1 );
2939 DebugSerial.println( " RPM/s" );
2940 DebugSerial.println( "[BOOT] PID: POSITIONAL + FEED-FORWARD + ANTI-WINDUP" );
2941 DebugSerial.println( "[BOOT] Stall latch: MANUAL MAIN OFF -> ON RESET" );
2942 DebugSerial.println( RUNNING_IN_WOKWI ? "[BOOT] INA219 source: INTERACTIVE WOKWI SLIDERS" : "[BOOT] INA219 source: PHYSICAL I2C" );
2943 DebugSerial.print( "[BOOT] Run profile: " );
2944 DebugSerial.println( RUNNING_IN_WOKWI ? "WOKWI SCALED RPM" : ( PHYSICAL_INITIAL_TEST_PROFILE ? "PHYSICAL INITIAL TEST" : "PHYSICAL R
ACE" ) );
2945 DebugSerial.print( "[BOOT] Normal target RPM: " );
2946 DebugSerial.println( NORMAL_BASE_RPM, 1 );
2947 DebugSerial.print( "[BOOT] Dynamic corner-speed test: " );
2948 DebugSerial.println( WOKWI_AUTO_CORNER_SPEED_TEST ? "ENABLED" : "DISABLED" );
2949 DebugSerial.println( "[BOOT] Wokwi test scenario: CORNER SPEED ONLY" );
2950 DebugSerial.print( "[BOOT] Autonomous bridge test: " );
2951 DebugSerial.println( WOKWI_AUTO_BRIDGE_TEST ? "ENABLED" : "DISABLED" );
```

Integrated autonomous controller

ORIGINAL LINES 2,952-2,990

```
2952     DebugSerial.println( "[BOOT] Bridge profile: MILD SLOPE / HIGHER SPEED" );
2953     DebugSerial.println();
2954     DebugSerial.println( "[READY] Starting autonomous control..." );
2955     DebugSerial.println();
2956 }
2957 // =====
2958 // LOOP
2959 // =====
2960 void loop() {
2961     // =====
2962     // EMERGENCY STOP IS CHECKED EVERY LOOP
2963     // =====
2964     readEmergencyStop();
2965     readMainSwitch();
2966     if (emergencyStopActive) {
2967         applyEmergencyStopImmediately();
2968         // Keep sampling encoders so diagnostics can show
2969         // the wheels winding down after the drive is cut.
2970         readLeftEncoder();
2971         readRightEncoder();
2972         printDiagnostics();
2973         return;
2974     }
2975     if ( !mainSwitchOn || !raceStartInterlockCleared ) {
2976         applySystemOffImmediately();
2977         // In Wokwi the MCU stays alive so we can display
2978         // SYSTEM OFF diagnostics.
2979         readLeftEncoder();
2980         readRightEncoder();
2981         printDiagnostics();
2982         return;
2983     }
2984     // E-Stop released + main switch ON:
2985     digitalWrite( STBY, HIGH );
2986     serviceMotors();
2987     updateWokwiBridgeScenario();
2988     // Read Raspberry Pi look-ahead commands as soon as possible.
2989     updateVisionSerial();
2990     // Fast steering / road-edge reaction
```

Integrated autonomous controller

ORIGINAL LINES 2,991-2,995

```
2991     updateFastRoadControl();
2992     // Slower wheel-speed PID + health/stability checks
2993     updatePID();
2994     printDiagnostics();
2995 }
```